Université / École Supérieure

Département d'Informatique

Filière : Génie Logiciel / Réseaux et Systèmes

Dossier Technique

Logiciel de Gestion des Opérations d'une Microfinance

Projet : **MicroFin Manager**

Type : Application Web de Gestion Bancaire

Technologies : Java Spring Boot, Oracle Database, HTML/CSS/PHP

Effectif : 5 personnes

Durée : 2 mois

Année : 2025 – 2026

Rôle	Membre
Chef de Projet	*Nom Prénom*
Admin Base de Données (Oracle)	*Nom Prénom*
Admin Système	*Nom Prénom*
Développeur 1	*Nom Prénom*
Développeur 2 / Gestionnaire	*Nom Prénom*

27 mars 2026

Table des matières

1 Introduction Générale **4**

1.1 Préambule 4

1.2 Problématique 4

1.3 Objectifs du Projet 5

1.3.1 Objectif général 5

1.3.2 Objectifs spécifiques 5

1.4 Périmètre du Dossier 5

2 Contexte et Objectifs **7**

2.1 Contexte Institutionnel 7

2.1.1 Le secteur de la microfinance 7

2.1.2 Les Systèmes Financiers Décentralisés (SFD) 7

2.2 Analyse de l'Existant 8

2.2.1 Situation actuelle 8

2.2.2 Limites identifiées 8

2.3 Cahier des Charges 9

2.3.1 Exigences fonctionnelles 9

2.3.2 Exigences non fonctionnelles 10

2.3.3 Contraintes du projet 10

3 Système d'Information **11**

3.1 Définition et Enjeux du SI 11

3.2 Cartographie du Système d'Information 11

3.2.1 Vue d'ensemble 11

3.2.2 SI Stratégique (Décisionnel) 12

3.2.3 SI Opérationnel (Transactionnel) 12

3.2.4 SI Réglementaire (Conformité) 12

3.2.5 SI Communication (Relationnel) 13

3.3 Flux d'Information 13

3.3.1 Flux principaux 13

3.3.2 Matrice des flux 13

3.4 Processus Métier 14
3.4.1 Processus d'ouverture de compte 14
3.4.2 Processus de versement 14
3.4.3 Processus de retrait 14
3.4.4 Processus de virement 14
3.5 Dictionnaire de Données 15
3.6 Règles de Gestion 15

4 Architecture Technique 17
4.1 Vue d'Ensemble de l'Architecture 17
4.2 Architecture en Couches Détaillée 17
4.2.1 Couche Présentation (Front-End) 17
4.2.2 Couche Métier (Back-End — Spring Boot) 19
4.2.3 Couche Données (Oracle Database) 21
4.3 Patterns Architecturaux 21
4.3.1 Pattern Repository 21
4.3.2 Pattern Service Layer 22
4.3.3 Pattern DTO (Data Transfer Object) 23
4.4 Communication entre Couches 24
4.4.1 API REST 24
4.4.2 Format des échanges 24

5 Technologies Utilisées — Décorticage Exhaustif 25
5.1 Langages de Programmation 25
5.1.1 Java (JDK 17+) 25
5.1.2 PHP 8+ 26
5.1.3 HTML5 27
5.1.4 CSS3 28
5.1.5 JavaScript (ES6+) 28
5.1.6 SQL et PL/SQL 29
5.1.7 Python 3.x 29
5.2 Frameworks et Bibliothèques 31
5.2.1 Spring Boot 3.x 31
5.2.2 Hibernate / JPA 33
5.2.3 Bootstrap 5 34
5.2.4 Autres bibliothèques 35
5.3 Système de Gestion de Base de Données 35
5.3.1 Oracle Database 35
5.4 Outils de Développement et de Build 37
5.5 Outils d'Infrastructure et Réseau 38

6 Modélisation UML **39**
6.1 Diagramme de Cas d'Utilisation . . . 39
6.1.1 Identification des acteurs . . . 40
6.1.2 Cas d'utilisation principaux . . . 41
6.1.3 Description détaillée d'un cas d'utilisation . . . 41
6.2 Diagramme de Classes . . . 42
6.2.1 Entités principales . . . 42
6.2.2 Relations entre classes . . . 43
6.3 Diagramme d'Activités . . . 43
6.3.1 Activité : Processus de virement . . . 43
6.4 Diagramme de Séquence . . . 44
6.4.1 Séquence : Retrait d'espèces . . . 44
6.5 Diagramme d'États . . . 45
6.5.1 États d'un Compte . . . 45
6.5.2 États d'un Client . . . 45
6.6 Diagramme de Composants et de Déploiement . . . 46
6.6.1 Composants logiciels . . . 46
6.6.2 Diagramme de déploiement . . . 46

7 Conception de la Base de Données Oracle **47**
7.1 Modèle Conceptuel de Données (MCD) . . . 47
7.1.1 Entités et attributs . . . 49
7.2 Modèle Logique de Données (MLD) . . . 56
7.2.1 Script DDL Oracle . . . 56
7.3 Procédures Stockées PL/SQL . . . 58
7.3.1 Calcul des agios . . . 58
7.3.2 Trigger d'audit . . . 59
7.4 Stratégie d'Indexation . . . 60
7.5 Politique de Sauvegarde . . . 61

8 Spécification Détaillée des Fonctionnalités **62**
8.1 Module 1 : Gestion des Clients . . . 62
8.1.1 Enregistrement d'un client . . . 62
8.1.2 Mise à jour des informations . . . 62
8.1.3 Recherche et consultation . . . 63
8.2 Module 2 : Gestion des Comptes . . . 63
8.2.1 Ouverture de compte . . . 63
8.2.2 Numérotation des comptes . . . 63
8.2.3 Suivi de compte . . . 63
8.2.4 Blocage et déblocage . . . 64

8.2.5 Fermeture de compte 64
8.3 Module 3 : Gestion des Opérations 65
8.3.1 Versement (Dépôt) 65
8.3.2 Retrait 65
8.3.3 Virement 66
8.3.4 Surveillance des opérations 66
8.4 Module 4 : Gestion des Finances 66
8.4.1 Conformité BCEAO 66
8.4.2 Calcul des AGIOS 66
8.4.3 SMS Banking 67
8.4.4 E-Banking 67
8.4.5 Carte VISA 67
8.4.6 Transactions avec contrôle 67

9 Infrastructure Réseau 69
9.1 Topologie du Réseau 69
9.1.1 Schéma de la topologie 69
9.2 Plan d'Adressage IP 70
9.2.1 Caractéristiques du réseau 70
9.3 Configuration du Serveur DHCP 70
9.3.1 ISC DHCP Server (Linux) 70
9.3.2 Windows Server DHCP (alternative) 71
9.4 Configuration du Serveur Applicatif 72
9.4.1 Apache HTTP Server 72
9.4.2 Tomcat embarqué (Spring Boot) 72
9.5 Configuration du Serveur Oracle 73
9.5.1 Oracle Net (Listener) 73
9.6 Sécurité Réseau 74
9.6.1 Règles de pare-feu 74
9.6.2 Configuration iptables 74

10 Méthodologie de Développement 76
10.1 Choix de la Méthodologie 76
10.1.1 Analyse comparative 76
10.1.2 Recommandation 77
10.2 Processus Unifié (UP) — Phases 77
10.2.1 Phase 1 : Inception (Semaines 1–2) 77
10.2.2 Phase 2 : Élaboration (Semaines 2–4) 78
10.2.3 Phase 3 : Construction (Semaines 4–7) 78
10.2.4 Phase 4 : Transition (Semaine 8) 79

10.3 Pratiques Scrum Intégrées . . . 79
10.3.1 Artefacts Scrum . . . 79
10.3.2 Cérémonies Scrum . . . 79
10.4 Gestion de Version (Git) . . . 80
10.4.1 Stratégie de branching . . . 80
10.4.2 Convention de commits . . . 80
10.5 Gestion de la Qualité . . . 81
10.5.1 Stratégie de test . . . 81
10.5.2 Revue de code . . . 81

11 Sécurité **82**
11.1 Politique de Sécurité Globale . . . 82
11.1.1 Principes directeurs . . . 82
11.2 Sécurité Applicative . . . 83
11.2.1 Authentification . . . 83
11.2.2 Autorisation (RBAC) . . . 84
11.2.3 Protection contre les vulnérabilités OWASP . . . 85
11.3 Sécurité de la Base de Données . . . 85
11.4 Sécurité Réseau . . . 86
11.5 Conformité et Audit . . . 86
11.5.1 Journal d'audit . . . 86
11.5.2 Traçabilité des opérations . . . 86

12 Plan de Déploiement **87**
12.1 Environnements . . . 87
12.2 Procédure de Déploiement . . . 87
12.2.1 Étape 1 : Préparation des serveurs . . . 87
12.2.2 Étape 2 : Déploiement du back-end . . . 88
12.2.3 Étape 3 : Déploiement du front-end . . . 89
12.2.4 Étape 4 : Peuplement de la base . . . 89
12.2.5 Étape 5 : Vérification post-déploiement . . . 90
12.3 Gestion des Mises à Jour . . . 90

13 Organisation de l'Équipe **91**
13.1 Composition de l'Équipe . . . 91
13.2 Matrice RACI . . . 91
13.3 Outils de Collaboration . . . 93
13.4 Répartition du Travail par Sprint . . . 93
13.4.1 Sprint 1 — Module Clients (Semaines 4–5) . . . 93
13.4.2 Sprint 2 — Module Comptes (Semaines 5–6) . . . 94

13.4.3 Sprint 3 — Module Opérations (Semaines 6–7) 94
13.4.4 Sprint 4 — Module Finances (Semaine 7–8) 95
13.5 Gestion des Risques 95

14 Livrables et Communication **96**
14.1 Liste des Livrables 97
14.2 Arborescence des Livrables 98
14.3 Communication du Projet 101
14.3.1 Logo 101
14.3.2 Messages de communication 101
14.4 Guide Utilisateur — Sommaire 101

15 Annexes **103**
15.1 Annexe A — Configuration Spring Boot complète 103
15.2 Annexe B — Script Python de génération de données 105
15.3 Annexe C — Exemples de requêtes API (Postman) 110
15.3.1 Authentification 110
15.3.2 Créer un client 110
15.3.3 Effectuer un virement 111
15.4 Annexe D — Glossaire 112
15.5 Annexe E — Bibliographie et Références 114

Table des figures

3.1 Les quatre piliers du SI de la microfinance 11
3.2 Flux d'information principaux 13

4.1 Architecture trois tiers du système 17
4.2 Pattern MVC dans Spring Boot 19

9.1 Topologie réseau du projet 69

Liste des tableaux

2.1 Formes juridiques des SFD 8
2.2 Analyse des limites de l'existant 8
2.3 Exigences non fonctionnelles 10

3.1 Composantes du SI Stratégique 12
3.2 Modules du SI Opérationnel 12
3.3 Matrice des flux d'information 13
3.4 Dictionnaire de données — Entités principales 15

4.1 Composants de la couche présentation 18
4.2 Rôle de chaque couche dans Spring Boot 21
4.3 Principaux endpoints de l'API REST 24

5.1 Fiche technique — Java 25
5.2 Fiche technique — PHP 26
5.3 Fiche technique — HTML5 27
5.4 Fiche technique — CSS3 28
5.5 Fiche technique — JavaScript 28
5.6 Fiche technique — SQL / PL/SQL 29
5.7 Fiche technique — Python 29
5.8 Fiche technique — Spring Boot 31
5.9 Modules Spring Boot du projet 31
5.10 Fiche technique — Hibernate / JPA 33
5.11 Fiche technique — Bootstrap 34
5.12 Bibliothèques complémentaires 35
5.13 Fiche technique — Oracle Database 35
5.14 Connecteurs Oracle 36
5.15 Outils de développement 37
5.16 Outils d'infrastructure 38

6.1 Acteurs du système 40
6.2 Cas d'utilisation par module 41

6.3 Classes principales du modèle de domaine 42
6.4 Relations du modèle de domaine 43
6.5 Séquence de messages — Retrait d'espèces 44
6.6 Transitions d'état d'un compte 45
6.7 Transitions d'état d'un client 45
6.8 Composants du système 46
6.9 Nœuds de déploiement 46

7.1 Entité CLIENT 49
7.2 Entité COMPTE 50
7.3 Entité OPERATION 51
7.4 Entité UTILISATEUR 52
7.5 Entité JOURNAL_AUDIT 53
7.6 Entité AGIOS 54
7.7 Entité CARTE_VISA 55
7.8 Entité NOTIFICATION 56
7.9 Stratégie d'indexation 60
7.10 Plan de sauvegarde Oracle 61

8.1 Fiche fonctionnelle — Enregistrement client 62
8.2 Types de comptes et caractéristiques 63
8.3 Motifs de blocage de compte 64
8.4 Fiche fonctionnelle — Versement 65
8.5 Fiche fonctionnelle — Retrait 65
8.6 Fiche fonctionnelle — Virement 66
8.7 Services SMS Banking 67

9.1 Plan d'adressage IP 70
9.2 Règles de pare-feu (iptables / firewalld) 74

10.1 Comparaison Processus Unifié vs Scrum 76
10.2 Activités de la phase Inception 77
10.3 Activités de la phase Élaboration 78
10.4 Activités de la phase Construction 78
10.5 Activités de la phase Transition 79
10.6 Cérémonies Scrum adaptées 79
10.7 Niveaux de test 81

11.1 Mécanismes d'authentification 83
11.2 Matrice des droits par rôle (RBAC) 84
11.3 Mesures contre les vulnérabilités OWASP Top 10 85

12.1 Environnements de déploiement 87
12.2 Checklist de vérification post-déploiement 90

13.1 Composition de l'équipe projet 91
13.2 Matrice RACI 92
13.3 Outils de collaboration de l'équipe 93
13.4 Répartition Sprint 1 — Module Clients 93
13.5 Répartition Sprint 2 — Module Comptes 94
13.6 Répartition Sprint 3 — Module Opérations 94
13.7 Répartition Sprint 4 — Module Finances 95
13.8 Registre des risques 95

14.1 Inventaire exhaustif des livrables 97
14.2 Messages de communication 101

Chapitre 1

Introduction Générale

1.1 Préambule

Le secteur de la microfinance constitue un pilier essentiel de l'inclusion financière dans les économies en développement. Les institutions de microfinance (IMF) jouent un rôle crucial en fournissant des services financiers — épargne, crédit, transferts, assurances — à des populations traditionnellement exclues du système bancaire classique. La gestion opérationnelle de ces structures repose sur des processus rigoureux de traitement des données clients, de suivi des comptes, de gestion des transactions et de conformité réglementaire, notamment vis-à-vis des normes édictées par la Banque Centrale des États de l'Afrique de l'Ouest (BCEAO).

Face à la complexité croissante de ces opérations et au volume grandissant des données à traiter (environ 150 clients et plus de 600 opérations dans le cadre de ce projet), l'informatisation devient non seulement un avantage concurrentiel mais une nécessité opérationnelle. Le présent projet vise à concevoir, développer et déployer un **logiciel de gestion des opérations d'une microfinance**, couvrant l'ensemble du cycle de vie des services financiers offerts par une IMF.

1.2 Problématique

Les institutions de microfinance font face à plusieurs défis majeurs dans leur gestion quotidienne :

1. **Gestion manuelle ou semi-manuelle** : de nombreuses IMF fonctionnent encore avec des registres papier ou des tableurs, sources d'erreurs, de lenteur et de perte de données.
2. **Traçabilité insuffisante** : l'absence de système centralisé rend difficile le suivi des opérations, la détection des anomalies et l'audit des transactions.
3. **Conformité réglementaire** : les exigences de la BCEAO imposent des normes strictes

en matière de reporting, de calcul des agios, de plafonds de transactions et de gestion des risques.

4. **Sécurité des données** : les informations financières et personnelles des clients nécessitent un niveau de protection élevé contre les accès non autorisés et les pertes de données.
5. **Scalabilité** : un système doit pouvoir supporter la croissance du portefeuille clients sans dégradation de performance.

1.3 Objectifs du Projet

1.3.1 Objectif général

Concevoir et réaliser une application web complète de gestion des opérations d'une institution de microfinance, intégrant les fonctionnalités bancaires essentielles, le respect des normes BCEAO, et une infrastructure réseau sécurisée.

1.3.2 Objectifs spécifiques

— Modéliser le Système d'Information (SI) de la microfinance à l'aide des diagrammes Unified Modeling Language (UML) (classes, cas d'utilisation, activités, séquence, états, composants et déploiement).
— Concevoir et implémenter une base de données relationnelle Oracle robuste et normalisée.
— Développer le back-end en `Java Spring Boot` avec une architecture Modèle-Vue-Contrôleur (MVC) respectant les principes Representational State Transfer (REST).
— Développer le front-end web en `HTML`, `CSS` et `PHP`.
— Implémenter les modules fonctionnels : gestion des clients, des comptes, des opérations et des finances.
— Mettre en place l'infrastructure réseau : serveur Dynamic Host Configuration Protocol (DHCP), adresses Internet Protocol (IP) fixes, machines clientes.
— Assurer la sécurité des données, l'authentification des utilisateurs et la gestion des droits d'accès.
— Générer les données de test (environ 150 clients, 600 opérations) via `Python` ou intelligence artificielle.
— Livrer une application fonctionnelle accompagnée d'une présentation intermédiaire.

1.4 Périmètre du Dossier

Ce dossier technique constitue le document de référence exhaustif du projet. Il couvre :

— L'**analyse du contexte** et la définition des objectifs (chapitre 2).
— La **description du système d'information** avec la cartographie complète du SI (chapitre 3).
— L'**architecture technique** détaillée de la solution (chapitre 4).
— Le **décorticage des technologies** utilisées : langages, frameworks, Système de Gestion de Base de Données (SGBD), outils (chapitre 5).
— La **modélisation UML** complète du système (chapitre 6).
— La **conception de la base de données** Oracle (chapitre 7).
— La **spécification des fonctionnalités** métier (chapitre 8).
— L'**infrastructure réseau** et la configuration serveur (chapitre 9).
— La **méthodologie de développement** adoptée (chapitre 10).
— Les aspects de **sécurité** (chapitre 11).
— Le plan de **déploiement** (chapitre 12).
— L'**organisation de l'équipe** et la répartition des rôles (chapitre 13).
— Les **livrables** attendus (chapitre 14).

Chapitre 2

Contexte et Objectifs

2.1 Contexte Institutionnel

2.1.1 Le secteur de la microfinance

La microfinance désigne l'ensemble des services financiers (épargne, crédit, assurance, transfert d'argent) destinés aux populations à faibles revenus, exclues du système bancaire traditionnel. En Afrique de l'Ouest, ce secteur est régulé par la BCEAO qui impose un cadre réglementaire strict à travers :

— La **loi portant réglementation des Systèmes Financiers Décentralisés** (SFD) dans les États membres de l'UEMOA.
— Les **instructions relatives aux normes prudentielles** applicables aux SFD.
— Les **obligations de reporting** périodique à la Banque Centrale.
— Les **plafonds de taux d'intérêt** (taux d'usure fixé par la BCEAO).
— Les **normes de provisionnement** des créances en souffrance.

2.1.2 Les Systèmes Financiers Décentralisés (SFD)

Un SFD est une institution qui collecte l'épargne et/ou accorde du crédit à des catégories de populations non desservies par les banques classiques. Les SFD se déclinent en plusieurs formes juridiques :

Table 2.1 – Formes juridiques des SFD

Type	Description
Coopérative d'épargne et de crédit	Structure mutualiste où les membres sont à la fois épargnants et emprunteurs
Association	Structure à but non lucratif offrant des services de microcrédit
Société de microfinance	Société à but lucratif agréée pour exercer des activités de microfinance
Projet/Programme	Volet microfinance intégré à un projet de développement

2.2 Analyse de l'Existant

2.2.1 Situation actuelle

La majorité des petites et moyennes IMF gèrent encore leurs opérations via :

— Des **registres papier** pour l'enregistrement des clients et des opérations quotidiennes.
— Des **tableurs Excel** pour le suivi comptable et la production de rapports.
— Des **outils bureautiques** pour la correspondance et la documentation.

2.2.2 Limites identifiées

Table 2.2 – Analyse des limites de l'existant

Domaine	Limite	Impact
Intégrité des données	Saisie manuelle sujette aux erreurs	Données incohérentes, doublons
Performance	Lenteur de traitement sur volume croissant	Attente des clients, congestion
Sécurité	Accès non contrôlé aux données	Risque de fraude et de fuite
Traçabilité	Absence de piste d'audit	Non-conformité réglementaire
Reporting	Production manuelle des rapports	Rapports tardifs, erreurs
Disponibilité	Données localisées sur un poste	Perte de données en cas de panne

2.3 Cahier des Charges

2.3.1 Exigences fonctionnelles

Le logiciel doit couvrir les domaines fonctionnels suivants :

1. **Gestion des clients**
   - Enregistrement complet des clients (état civil, pièce d'identité, coordonnées, photo)
   - Mise à jour des informations personnelles
   - Recherche et consultation des fiches clients
   - Historique complet des opérations par client
   - Gestion des données personnelles conforme à la réglementation
2. **Gestion des comptes**
   - Ouverture de compte (épargne, courant, dépôt à terme)
   - Suivi de l'état et du solde des comptes
   - Blocage / déblocage de compte
   - Fermeture de compte avec solde de tout compte
   - Attribution automatique des numéros de compte
3. **Gestion des opérations**
   - Virements entre comptes (internes et externes)
   - Versements (dépôts d'espèces)
   - Retraits avec contrôle de solde
   - Surveillance des transactions (plafonds, seuils d'alerte)
   - Définition des montants maximaux et minimaux par type d'opération
4. **Gestion des finances**
   - Conformité aux options BCEAO
   - Système financier décentralisé
   - Calcul et application des AGIOS (intérêts débiteurs)
   - SMS Banking et E-Banking
   - Gestion des cartes VISA
   - Transactions avec contrôle et validation

2.3.2 Exigences non fonctionnelles

Table 2.3 – Exigences non fonctionnelles

Catégorie	Exigence	Priorité
Performance	Temps de réponse < 3 secondes pour les opérations courantes	Haute
Disponibilité	Système disponible 99% du temps pendant les heures ouvrées	Haute
Sécurité	Authentification forte, chiffrement des données sensibles	Critique
Scalabilité	Support de 150+ clients et 600+ opérations	Moyenne
Ergonomie	Interface intuitive, responsive, multilingue (français)	Haute
Maintenabilité	Code documenté, architecture modulaire	Moyenne
Auditabilité	Journalisation complète des actions utilisateurs	Haute
Interopérabilité	API REST pour intégration tierce	Moyenne

2.3.3 Contraintes du projet

— **Effectif** : équipe de 5 personnes aux rôles définis.
— **Délai** : 2 mois de développement.
— **Infrastructure** : 1 serveur applicatif, 1 serveur de base de données, 1 serveur DHCP, 2 machines clientes.
— **Technologies imposées** : Java Spring Boot, Oracle Database, HTML/CSS/PHP.
— **Données** : génération de 150 clients et 600 opérations via Python/IA.
— **Livrables** : présentation intermédiaire + application web fonctionnelle.
— **Communication** : logo du projet + 2 à 3 messages de communication.

Chapitre 3

Système d'Information

3.1 Définition et Enjeux du SI

Le SI d'une institution de microfinance constitue l'ensemble organisé des ressources (matérielles, logicielles, humaines, données, procédures) permettant de collecter, stocker, traiter et diffuser l'information nécessaire au fonctionnement de l'organisation. Dans le contexte d'une microfinance, le SI revêt une importance capitale car il conditionne :

— La **fiabilité des opérations financières** (chaque centime doit être tracé).
— La **conformité réglementaire** vis-à-vis des autorités de supervision (BCEAO).
— La **qualité du service client** (rapidité, exactitude).
— La **prise de décision managériale** (tableaux de bord, indicateurs).
— La **pérennité de l'institution** (gestion des risques, provisionnement).

3.2 Cartographie du Système d'Information

3.2.1 Vue d'ensemble

Le SI du projet **MicroFin Manager** s'articule autour de quatre piliers fondamentaux :

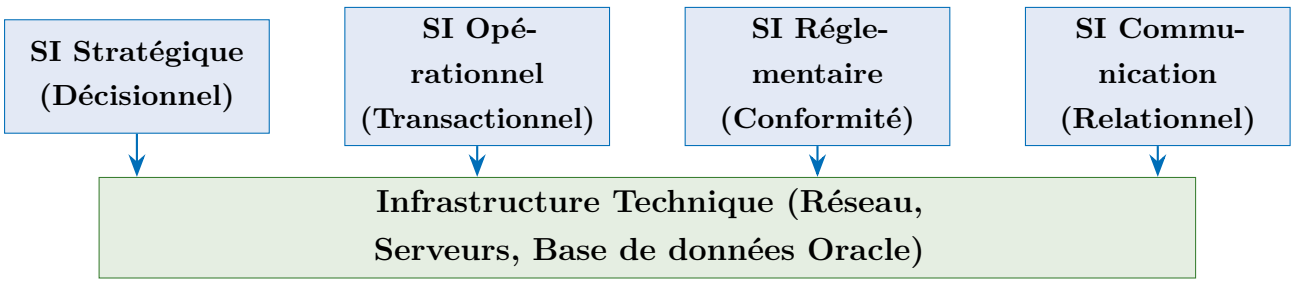

FIGURE 3.1 – Les quatre piliers du SI de la microfinance

3.2.2 SI Stratégique (Décisionnel)

Le SI stratégique fournit aux dirigeants les informations nécessaires à la prise de décision :

Table 3.1 – Composantes du SI Stratégique

Composante	Description
Tableaux de bord	Indicateurs clés de performance : nombre de clients actifs, volume des dépôts, encours de crédit, taux de remboursement, ratio de liquidité
Reporting	Rapports périodiques pour la direction et les autorités de tutelle
Analyse financière	Calcul des ratios prudentiels, suivi de la rentabilité par produit
Gestion des risques	Identification et suivi des risques de crédit, de liquidité, opérationnels

3.2.3 SI Opérationnel (Transactionnel)

C'est le cœur du système, traitant les opérations quotidiennes :

Table 3.2 – Modules du SI Opérationnel

Module	Fonctions	Criticité
Gestion des Clients	Création, modification, recherche, KYC (Know Your Customer)	Haute
Gestion des Comptes	Ouverture, suivi, blocage, clôture, relevés	Critique
Gestion des Opérations	Versements, retraits, virements, contrôles	Critique
Gestion des Finances	Agios, intérêts, commissions, frais	Haute
Gestion de la Trésorerie	Suivi de caisse, rapprochement, bilans journaliers	Haute

3.2.4 SI Réglementaire (Conformité)

Ce sous-système assure le respect des normes et obligations légales :

- **Normes BCEAO** : respect des ratios prudentiels (ratio de solvabilité, ratio de liquidité, limitation des risques).
- **Lutte anti-blanchiment (LAB)** : identification des opérations suspectes, déclaration de soupçon.
- **Reporting réglementaire** : états financiers, états statistiques, déclarations périodiques.
- **Protection des données personnelles** : conformité aux lois sur la protection des données.
- **Plafonds et seuils** : respect des montants maximaux et minimaux par type d'opération.

3.2.5 SI Communication (Relationnel)

Ce sous-système gère la relation avec les clients et les partenaires :

— **SMS Banking** : notifications par SMS, alertes de solde, confirmations d'opérations.
— **E-Banking** : portail web client pour consultation de solde et historique.
— **Carte VISA** : interface avec les réseaux de paiement internationaux.
— **Communication institutionnelle** : messages promotionnels, informations réglementaires.

3.3 Flux d'Information

3.3.1 Flux principaux

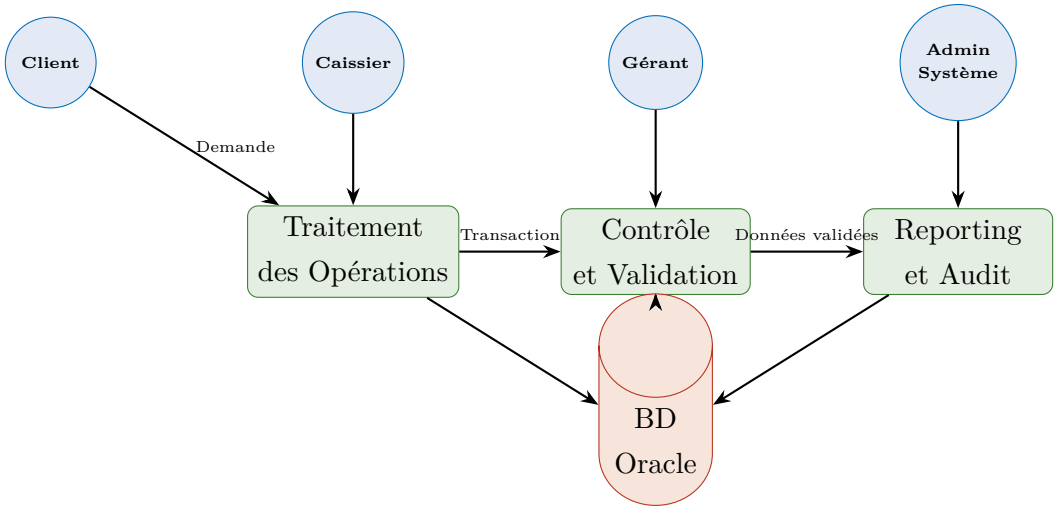

FIGURE 3.2 – Flux d'information principaux

3.3.2 Matrice des flux

TABLE 3.3 – Matrice des flux d'information

Émetteur	Récepteur	Type	Données échangées
Client	Caissier	Physique	Demande d'opération, pièce d'identité, espèces
Caissier	Système	Numérique	Saisie d'opération, identification client
Système	Base Oracle	Numérique	Requêtes SQL, transactions ACID
Système	Caissier	Numérique	Confirmation, reçu, alerte
Système	Client	Numérique	SMS, relevé, notification E-Banking
Système	BCEAO	Numérique	Rapports réglementaires, états financiers
Admin Système	Serveurs	Numérique	Configuration, maintenance, sauvegarde
Gérant	Système	Numérique	Validation d'opérations exceptionnelles

3.4 Processus Métier

3.4.1 Processus d'ouverture de compte

1. Le client se présente avec ses pièces d'identité.
2. Le gestionnaire vérifie l'identité et la conformité KYC.
3. Le gestionnaire saisit les informations du client dans le système.
4. Le système attribue automatiquement un numéro de compte.
5. Le client effectue un premier versement (montant minimum requis).
6. Le système génère le contrat d'ouverture de compte.
7. Le client signe le contrat et reçoit ses documents.
8. Le système enregistre l'opération et envoie un SMS de confirmation.

3.4.2 Processus de versement

1. Le client se présente au guichet avec les espèces.
2. Le caissier identifie le client et son compte.
3. Le caissier saisit le montant du versement.
4. Le système vérifie la conformité (plafonds, seuils LAB).
5. Le système crédite le compte et enregistre l'opération.
6. Le système génère un reçu et envoie un SMS de confirmation.

3.4.3 Processus de retrait

1. Le client se présente au guichet avec une pièce d'identité.
2. Le caissier identifie le client et vérifie le solde disponible.
3. Le caissier saisit le montant du retrait.
4. Le système vérifie : solde suffisant, montant dans les limites autorisées, compte non bloqué.
5. Si validation : le système débite le compte et génère un reçu.
6. Si refus : le système affiche le motif (solde insuffisant, compte bloqué, plafond dépassé).
7. SMS de confirmation envoyé au client.

3.4.4 Processus de virement

1. Le client demande un virement (source, destination, montant).
2. Le système vérifie l'existence des deux comptes.
3. Le système vérifie le solde du compte source.
4. Le système vérifie les plafonds de virement.
5. En cas de validation : débit du compte source, crédit du compte destination (transaction atomique).

6. Enregistrement de l'opération dans le journal des transactions.
7. Notification aux deux parties (SMS/E-Banking).

3.5 Dictionnaire de Données

TABLE 3.4 – Dictionnaire de données — Entités principales

Donnée	Type	Taille	Description
id_client	NUMBER	10	Identifiant unique du client (séquence Oracle)
nom	VARCHAR2	100	Nom de famille du client
prenom	VARCHAR2	100	Prénom(s) du client
date_naissance	DATE	—	Date de naissance
num_piece	VARCHAR2	30	Numéro de pièce d'identité
type_piece	VARCHAR2	20	CNI, Passeport, Permis conduire
telephone	VARCHAR2	20	Numéro de téléphone principal
email	VARCHAR2	150	Adresse email
adresse	VARCHAR2	255	Adresse postale complète
photo	BLOB	—	Photo d'identité du client
date_creation	TIMESTAMP	—	Date et heure de création du dossier
statut_client	VARCHAR2	20	ACTIF, INACTIF, SUSPENDU
num_compte	VARCHAR2	20	Numéro de compte unique
type_compte	VARCHAR2	30	EPARGNE, COURANT, DAT
solde	NUMBER	15,2	Solde courant du compte
statut_compte	VARCHAR2	20	ACTIF, BLOQUE, FERME
id_operation	NUMBER	15	Identifiant unique de l'opération
type_operation	VARCHAR2	30	VERSEMENT, RETRAIT, VIREMENT
montant	NUMBER	15,2	Montant de l'opération
date_operation	TIMESTAMP	—	Date et heure de l'opération

3.6 Règles de Gestion

Règles de gestion principales

RG1 : Un client peut posséder un ou plusieurs comptes.

RG2 : Un compte appartient à un et un seul client.

RG3 : Chaque opération est rattachée à un et un seul compte.

RG4 : Le solde d'un compte ne peut jamais devenir négatif (sauf autorisation de découvert).

RG5 : Toute opération supérieure à un seuil défini nécessite la validation d'un superviseur.

RG6 : Les virements sont des transactions atomiques : débit et crédit simultanés.
RG7 : Un compte bloqué ne peut faire l'objet d'aucune opération sauf déblocage.
RG8 : La fermeture d'un compte nécessite un solde nul.
RG9 : Les agios sont calculés quotidiennement sur les soldes débiteurs.
RG10 : Chaque opération génère un enregistrement dans le journal d'audit.
RG11 : Les taux d'intérêt sont conformes aux plafonds BCEAO.
RG12 : Le montant maximal d'un retrait quotidien est paramétrable.
RG13 : Le montant minimal d'ouverture de compte est paramétrable.
RG14 : Tout accès au système nécessite une authentification préalable.
RG15 : Les mots de passe sont stockés sous forme hachée (non réversible).

Chapitre 4

Architecture Technique

4.1 Vue d'Ensemble de l'Architecture

L'architecture du projet **MicroFin Manager** repose sur un modèle **trois tiers** (3-tier) classique dans le développement d'applications d'entreprise, adapté aux contraintes du projet :

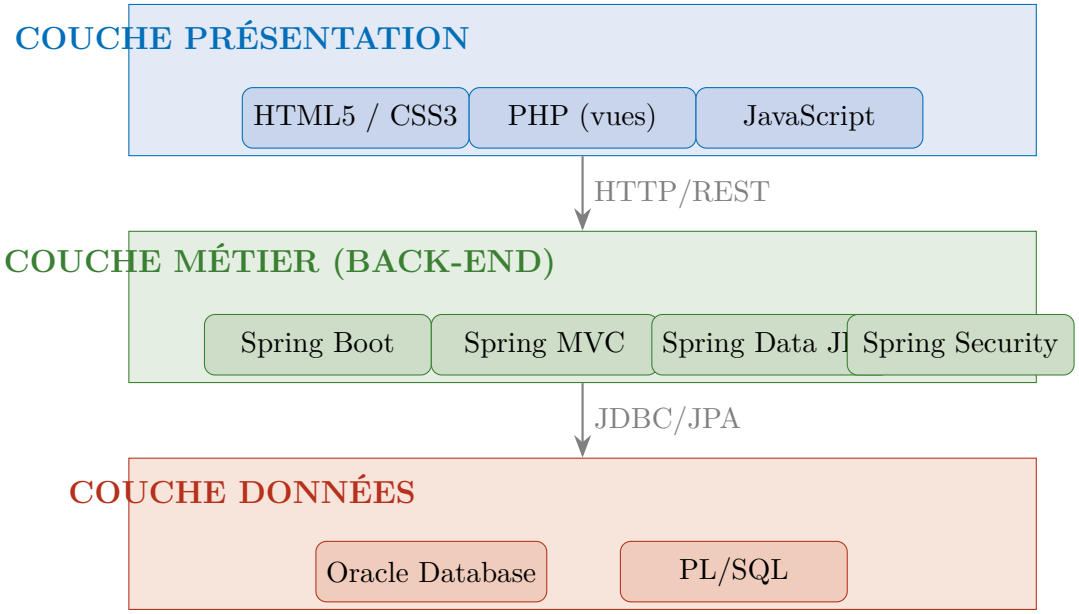

FIGURE 4.1 – Architecture trois tiers du système

4.2 Architecture en Couches Détaillée

4.2.1 Couche Présentation (Front-End)

La couche de présentation est responsable de l'interface utilisateur et de l'interaction avec les utilisateurs finaux.

Table 4.1 – Composants de la couche présentation

Technologie	Rôle	Détail
HTML5	Structure	Balisage sémantique des pages, formulaires, tableaux
CSS3	Style	Mise en forme, responsive design, animations
PHP	Rendu serveur	Génération dynamique des vues, gestion des sessions
JavaScript	Interactivité	Validation côté client, requêtes AJAX, DOM
Bootstrap	Framework CSS	Composants UI, grille responsive, thèmes

Structure des vues

```
webapp/
|-- index.php              # Page d'accueil / Login
|-- assets/
|   |-- css/
|   |   |-- style.css      # Feuille de style principale
|   |   |-- responsive.css
|   |-- js/
|   |   |-- main.js        # Scripts principaux
|   |   |-- validation.js # Validation des formulaires
|   |-- images/
|       |-- logo.png
|-- views/
|   |-- clients/
|   |   |-- liste.php
|   |   |-- creation.php
|   |   |-- modification.php
|   |   |-- fiche.php
|   |-- comptes/
|   |   |-- liste.php
|   |   |-- ouverture.php
|   |   |-- suivi.php
|   |-- operations/
|   |   |-- versement.php
|   |   |-- retrait.php
|   |   |-- virement.php
|   |   |-- historique.php
|   |-- finances/
|   |   |-- agios.php
|   |   |-- rapports.php
|-- includes/
```

```
|    |-- header.php
|    |-- footer.php
|    |-- sidebar.php
|    |-- config.php          # Configuration BD
```

Listing 4.1 – Arborescence du front-end

4.2.2 Couche Métier (Back-End — Spring Boot)

La couche métier constitue le cœur applicatif. Elle implémente la logique métier, les règles de gestion et orchestre les interactions entre la présentation et les données.

Architecture MVC de Spring Boot

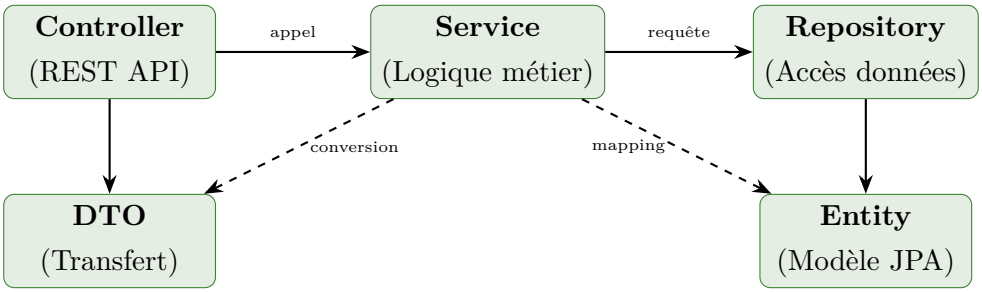

FIGURE 4.2 – Pattern MVC dans Spring Boot

Structure du projet Spring Boot

```
microfinance-api/
|-- src/main/java/com/microfinance/
|    |-- MicrofinanceApplication.java
|    |-- config/
|    |    |-- SecurityConfig.java
|    |    |-- CorsConfig.java
|    |    |-- SwaggerConfig.java
|    |-- controller/
|    |    |-- ClientController.java
|    |    |-- CompteController.java
|    |    |-- OperationController.java
|    |    |-- FinanceController.java
|    |    |-- AuthController.java
|    |-- service/
|    |    |-- ClientService.java
|    |    |-- CompteService.java
|    |    |-- OperationService.java
```

```
|   |   |-- FinanceService.java
|   |   |-- NotificationService.java
|   |-- repository/
|   |   |-- ClientRepository.java
|   |   |-- CompteRepository.java
|   |   |-- OperationRepository.java
|   |-- model/entity/
|   |   |-- Client.java
|   |   |-- Compte.java
|   |   |-- Operation.java
|   |   |-- Utilisateur.java
|   |-- model/dto/
|   |   |-- ClientDTO.java
|   |   |-- CompteDTO.java
|   |   |-- OperationDTO.java
|   |-- model/enums/
|   |   |-- TypeCompte.java
|   |   |-- TypeOperation.java
|   |   |-- StatutCompte.java
|   |-- exception/
|   |   |-- ResourceNotFoundException.java
|   |   |-- InsufficientBalanceException.java
|   |   |-- AccountBlockedException.java
|   |   |-- GlobalExceptionHandler.java
|   |-- security/
|   |   |-- JwtTokenProvider.java
|   |   |-- JwtAuthFilter.java
|   |   |-- UserDetailsServiceImpl.java
|   |-- util/
|       |-- AccountNumberGenerator.java
|       |-- AgioCalculator.java
|-- src/main/resources/
|   |-- application.properties
|   |-- application-dev.properties
|   |-- application-prod.properties
|-- pom.xml
```

Listing 4.2 – Arborescence du back-end Spring Boot

Couches logicielles internes

TABLE 4.2 – Rôle de chaque couche dans Spring Boot

Couche	Package	Responsabilité
Controller	`controller`	Réception des requêtes HTTP, validation des entrées, appel des services, retour des réponses JSON
Service	`service`	Implémentation de la logique métier, orchestration des appels aux repositories, gestion des transactions
Repository	`repository`	Accès aux données via JPA/Hibernate, requêtes SQL personnalisées
Entity	`model/entity`	Mapping objet-relationnel (ORM), annotations JPA, contraintes de validation
DTO	`model/dto`	Objets de transfert de données entre couches, sérialisation JSON
Config	`config`	Configuration Spring (sécurité, CORS, Swagger, sources de données)
Security	`security`	Authentification JWT, filtres de sécurité, gestion des rôles
Exception	`exception`	Gestion centralisée des erreurs, codes de retour HTTP

4.2.3 Couche Données (Oracle Database)

La couche données est gérée par le SGBD Oracle Database, garantissant :

— **Intégrité transactionnelle (ACID)** : Atomicité, Cohérence, Isolation, Durabilité pour chaque opération financière.
— **Performance** : indexation avancée, optimiseur de requêtes, mise en cache.
— **Procedural Language/SQL (PL/SQL)** : procédures stockées, fonctions, triggers pour les traitements côté serveur.
— **Sécurité** : chiffrement TDE (Transparent Data Encryption), audit natif.
— **Sauvegarde et restauration** : RMAN (Recovery Manager) pour les sauvegardes incrémentales.

4.3 Patterns Architecturaux

4.3.1 Pattern Repository

Le pattern Repository abstrait l'accès aux données, isolant la logique métier de la persistance :

```
@Repository
public interface ClientRepository
```

```
        extends JpaRepository<Client, Long> {

    Optional<Client> findByNumPiece(String numPiece);

    List<Client> findByStatut(StatutClient statut);

    @Query("SELECT c FROM Client c WHERE "
         + "LOWER(c.nom) LIKE LOWER(CONCAT('%', :q, '%')) "
         + "OR LOWER(c.prenom) LIKE LOWER(CONCAT('%', :q, '%'))")
    List<Client> rechercher(@Param("q") String query);

    @Query("SELECT COUNT(c) FROM Client c "
         + "WHERE c.statut = 'ACTIF'")
    long countClientsActifs();
}
```

Listing 4.3 – Exemple de Repository Spring Data JPA

4.3.2 Pattern Service Layer

La couche Service encapsule la logique métier et la gestion transactionnelle :

```
@Service
@Transactional
public class OperationService {

    @Autowired
    private CompteRepository compteRepo;

    @Autowired
    private OperationRepository operationRepo;

    public Operation effectuerRetrait(Long compteId,
                                      BigDecimal montant) {
        Compte compte = compteRepo.findById(compteId)
            .orElseThrow(() ->
                new ResourceNotFoundException("Compte introuvable
                    "));

        if (compte.getStatut() == StatutCompte.BLOQUE) {
            throw new AccountBlockedException(
                "Compte bloque, operation impossible");
        }
```

```
        if (compte.getSolde().compareTo(montant) < 0) {
            throw new InsufficientBalanceException(
                "Solde insuffisant");
        }

        compte.setSolde(
            compte.getSolde().subtract(montant));
        compteRepo.save(compte);

        Operation op = new Operation();
        op.setCompte(compte);
        op.setType(TypeOperation.RETRAIT);
        op.setMontant(montant);
        op.setDateOperation(LocalDateTime.now());
        return operationRepo.save(op);
    }
}
```

Listing 4.4 – Exemple de Service transactionnel

4.3.3 Pattern DTO (Data Transfer Object)

Les DTOs séparent la représentation interne des entités de leur exposition via l'API :

```
public class CompteDTO {
    private Long id;
    private String numCompte;
    private String typeCompte;
    private BigDecimal solde;
    private String statut;
    private String nomClient;
    private String prenomClient;
    private LocalDateTime dateOuverture;

    // Getters et Setters ...
}
```

Listing 4.5 – Exemple de DTO

4.4 Communication entre Couches

4.4.1 API REST

L'application expose une API RESTful pour la communication entre le front-end et le back-end :

TABLE 4.3 – Principaux endpoints de l'API REST

Méthode	Endpoint	Action	Description
GET	/api/clients	Liste	Récupérer tous les clients
GET	/api/clients/{id}	Détail	Récupérer un client par ID
POST	/api/clients	Créer	Enregistrer un nouveau client
PUT	/api/clients/{id}	Modifier	Mettre à jour un client
POST	/api/comptes	Ouvrir	Ouvrir un nouveau compte
PUT	/api/comptes/{id}/bloquer	Bloquer	Bloquer un compte
PUT	/api/comptes/{id}/debloquer	Débloquer	Débloquer un compte
POST	/api/operations/versement	Verser	Effectuer un versement
POST	/api/operations/retrait	Retirer	Effectuer un retrait
POST	/api/operations/virement	Virer	Effectuer un virement
GET	/api/operations/compte/{id}	Historique	Historique des opérations
POST	/api/auth/login	Login	Authentification

4.4.2 Format des échanges

Toutes les communications utilisent le format JSON :

```
{
    "id": 1,
    "numCompte": "MFN-2026-000001",
    "typeCompte": "EPARGNE",
    "solde": 250000.00,
    "statut": "ACTIF",
    "nomClient": "DIALLO",
    "prenomClient": "Mamadou",
    "dateOuverture": "2026-01-15T10:30:00"
}
```

Listing 4.6 – Exemple de réponse JSON

Chapitre 5

Technologies Utilisées — Décorticage Exhaustif

Ce chapitre présente un décorticage détaillé de chaque technologie, outil et langage de programmation utilisé dans le projet. Pour chaque élément, nous précisons sa nature, son rôle dans le projet, ses caractéristiques techniques et sa justification.

5.1 Langages de Programmation

5.1.1 Java (JDK 17+)

Table 5.1 – Fiche technique — Java

Propriété	Valeur
Nature	Langage de programmation orienté objet, compilé puis interprété (bytecode JVM)
Version recommandée	JDK 17 LTS (Long Term Support)
Paradigme	Orienté objet, fonctionnel (depuis Java 8+)
Typage	Statique, fort
Exécution	Compilé en bytecode, exécuté par la JVM
Licence	GPL v2 (OpenJDK)

Caractéristiques clés utilisées

— **Portabilité (WORA — Write Once, Run Anywhere)** : le bytecode Java s'exécute sur toute plateforme disposant d'une JVM, garantissant l'indépendance vis-à-vis du système d'exploitation du serveur.

— **Gestion automatique de la mémoire** : le Garbage Collector (GC) libère automatiquement les objets non référencés, évitant les fuites mémoire critiques dans une

application financière tournant en continu.

— **Programmation orientée objet** : encapsulation, héritage, polymorphisme et abstraction pour modéliser fidèlement les entités métier (Client, Compte, Opération).
— **API Collections** : `List`, `Map`, `Set`, `Queue` pour la manipulation efficace des données en mémoire.
— **Streams API (Java 8+)** : traitement fonctionnel des collections (filtrage, mapping, réduction) pour les traitements batch.
— **Optional** : gestion explicite des valeurs nulles, réduisant les `NullPointerException`.
— **Records (Java 14+)** : classes immuables simplifiées, idéales pour les DTOs.
— **Try-with-resources** : gestion automatique de la fermeture des ressources (Java DataBase Connectivity (JDBC), fichiers).
— **Annotations** : métadonnées déclaratives (`@Entity`, `@Autowired`, `@Transactional`).
— **Multithreading** : support natif de la programmation concurrente pour les traitements parallèles (calcul des agios, envoi de SMS).

Rôle dans le projet

Java est le langage **principal** du back-end. Il est utilisé pour :

— L'implémentation de toute la logique métier (services, règles de gestion).
— L'exposition de l'API REST via Spring Boot.
— Le mapping objet-relationnel avec Hibernate/JPA.
— La sécurité applicative (authentification JWT, contrôle d'accès).
— Les traitements transactionnels (virements atomiques, calcul d'agios).

5.1.2 PHP 8+

Table 5.2 – Fiche technique — PHP

Propriété	Valeur
Nature	Langage de script côté serveur, interprété
Version recommandée	PHP 8.1+
Paradigme	Multi-paradigme (procédural, OO, fonctionnel)
Typage	Dynamique (typage strict optionnel depuis PHP 7)
Exécution	Interprété par le moteur Zend
Licence	PHP License

Caractéristiques clés utilisées

— **Intégration HTML native** : PHP s'insère directement dans le code HTML via les balises `<?php ?>`, simplifiant la génération dynamique de pages.
— **Gestion des sessions** : `$_SESSION` pour maintenir l'état de connexion des utilisateurs.

— **cURL / file_get_contents** : communication avec l'API Spring Boot en HTTP.
— **PDO (PHP Data Objects)** : accès unifié aux bases de données (utilisé pour d'éventuelles requêtes directes).
— **Fonctions de hachage** : `password_hash()` et `password_verify()` pour la sécurité côté front.
— **Filtres et validation** : `filter_var()`, `htmlspecialchars()` pour la sécurisation des entrées.

Rôle dans le projet

PHP sert de **couche de présentation serveur** :
— Génération dynamique des pages HTML.
— Gestion des sessions utilisateur et de l'authentification côté front.
— Consommation de l'API REST Spring Boot (via cURL).
— Rendu des formulaires, tableaux et interfaces utilisateur.

5.1.3 HTML5

Table 5.3 – Fiche technique — HTML5

Propriété	Valeur
Nature	Langage de balisage structurel pour le web
Version	HTML5 (standard W3C/WHATWG)
Rôle	Structure et sémantique des pages web

Éléments HTML5 utilisés

— **Balises sémantiques** : `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>` pour une structure accessible.
— **Formulaires avancés** : `<input type="email">`, `<input type="tel">`, `<input type="date">`, `<input type="number">` avec validation native du navigateur.
— **Tableaux** : `<table>`, `<thead>`, `<tbody>` pour l'affichage des listes de clients, comptes, opérations.
— **Attributs data-*** : stockage de métadonnées dans le DOM pour JavaScript.
— **Accessibilité** : attributs ARIA (`role`, `aria-label`) pour l'accessibilité.

5.1.4 CSS3

Table 5.4 – Fiche technique — CSS3

Propriété	Valeur
Nature	Langage de feuilles de style en cascade
Version	CSS3 (modules W3C)
Rôle	Mise en forme visuelle des pages web

Fonctionnalités CSS3 exploitées

— **Flexbox** : disposition flexible des éléments dans les formulaires et tableaux de bord.
— **Grid Layout** : grille bidimensionnelle pour les mises en page complexes.
— **Media Queries** : adaptation responsive — desktop, tablette, mobile.
— **Variables CSS** : `-primary-color`, `-accent-color` pour la cohérence chromatique.
— **Transitions et animations** : effets visuels sur les interactions (hover, focus).
— **Box-shadow et border-radius** : esthétique moderne des cartes et boutons.
— **Pseudo-classes** : `:hover`, `:focus`, `:nth-child()` pour le style dynamique des tableaux.

5.1.5 JavaScript (ES6+)

Table 5.5 – Fiche technique — JavaScript

Propriété	Valeur
Nature	Langage de script côté client (et serveur)
Version	ECMAScript 2015+ (ES6+)
Paradigme	Multi-paradigme (procédural, OO, fonctionnel, événementiel)
Typage	Dynamique, faible
Exécution	Interprété par le moteur du navigateur (V8, SpiderMonkey)

Utilisation dans le projet

— **Validation des formulaires** : vérification en temps réel des champs (format email, montants, dates) avant soumission.
— **Requêtes AJAX (Fetch API)** : communication asynchrone avec l'API REST sans rechargement de page.
— **Manipulation du DOM** : mise à jour dynamique de l'interface (notifications, alertes, soldes).
— **Gestion des événements** : `addEventListener` pour les interactions utilisateur.
— **Formatage des données** : formatage des montants, dates et numéros de compte.

5.1.6 SQL et PL/SQL

Table 5.6 – Fiche technique — SQL / PL/SQL

Propriété	Valeur
Nature	Langage de requête (SQL) et langage procédural (PL/SQL)
Standard	SQL :2016 (Oracle supporte la majorité du standard)
PL/SQL	Extension procédurale propriétaire Oracle
Rôle	Manipulation, définition et contrôle des données

Utilisation SQL

- **DDL** (Data Definition Language) : `CREATE TABLE`, `ALTER TABLE`, `DROP`, `CREATE INDEX`, `CREATE SEQUENCE`.
- **DML** (Data Manipulation Language) : `INSERT`, `UPDATE`, `DELETE`, `SELECT`.
- **DCL** (Data Control Language) : `GRANT`, `REVOKE` pour les permissions.
- **TCL** (Transaction Control Language) : `COMMIT`, `ROLLBACK`, `SAVEPOINT`.
- **Jointures** : `INNER JOIN`, `LEFT JOIN`, `OUTER JOIN` pour les requêtes multi-tables.
- **Fonctions d'agrégation** : `SUM`, `COUNT`, `AVG`, `MAX`, `MIN` pour les rapports financiers.
- **Sous-requêtes** : requêtes imbriquées pour les traitements complexes.

Utilisation PL/SQL

- **Procédures stockées** : traitements complexes exécutés côté serveur de base de données (calcul des agios, clôture journalière).
- **Fonctions** : calculs réutilisables (intérêts, commissions, conversion de devises).
- **Triggers** : déclencheurs automatiques sur les événements de la base de données (audit, mise à jour de solde, vérification de plafond).
- **Packages** : regroupement logique de procédures et fonctions par domaine métier.
- **Curseurs** : parcours ligne par ligne pour les traitements batch (relevés, rapports).
- **Gestion d'exceptions** : traitement des erreurs avec `EXCEPTION WHEN`.

5.1.7 Python 3.x

Table 5.7 – Fiche technique — Python

Propriété	Valeur
Nature	Langage de script interprété, multi-paradigme
Version	Python 3.10+
Rôle dans le projet	Génération de données de test
Bibliothèques	Faker, cx_Oracle, random, csv

Utilisation dans le projet

Python est utilisé exclusivement pour la **génération de données de test** :

- **Faker** : génération de données réalistes (noms, prénoms, adresses, numéros de téléphone, emails) adaptées au contexte africain francophone.
- **cx_Oracle** : connexion directe à la base Oracle pour l'insertion des données générées.
- **random** : génération de montants, dates et types d'opérations aléatoires mais cohérents.
- **csv** : export/import de données en format CSV comme alternative à l'insertion directe.
- Génération de **150 clients** avec données d'état civil complètes.
- Génération de **600 opérations** réparties sur les différents types (versements, retraits, virements).

```python
from faker import Faker
import cx_Oracle
import random
from datetime import datetime, timedelta

fake = Faker('fr_FR')

def generer_clients(n=150):
    clients = []
    for i in range(n):
        client = {
            'nom': fake.last_name().upper(),
            'prenom': fake.first_name(),
            'date_naissance': fake.date_of_birth(
                minimum_age=18, maximum_age=70),
            'telephone': f"+225 {random.randint(01,99):02d} "
                         f"{random.randint(0,99):02d} "
                         f"{random.randint(0,99):02d} "
                         f"{random.randint(0,99):02d}",
            'email': fake.email(),
            'adresse': fake.address(),
            'num_piece': f"CI{random.randint(100000,999999)}",
            'type_piece': random.choice(
                ['CNI', 'PASSEPORT', 'PERMIS']),
        }
        clients.append(client)
    return clients
```

Listing 5.1 – Exemple de script de génération de données

5.2 Frameworks et Bibliothèques

5.2.1 Spring Boot 3.x

TABLE 5.8 – Fiche technique — Spring Boot

Propriété	Valeur
Nature	Framework Java d'application d'entreprise
Version	Spring Boot 3.x (basé sur Spring Framework 6.x)
Éditeur	VMware (Broadcom) / Pivotal
Licence	Apache License 2.0
Serveur embarqué	Apache Tomcat (par défaut)
Gestionnaire de build	Apache Maven ou Gradle

Modules Spring utilisés

TABLE 5.9 – Modules Spring Boot du projet

Module	Fonction
`spring-boot-starter-web`	Serveur HTTP embarqué (Tomcat), contrôleurs REST, sérialisation JSON (Jackson)
`spring-boot-starter-data-jpa`	Abstraction JPA/Hibernate pour l'ORM, repositories auto-générés
`spring-boot-starter-security`	Authentification, autorisation, filtres de sécurité, protection CSRF
`spring-boot-starter-validation`	Validation des entrées via Bean Validation (JSR 380) : `@NotNull`, `@Size`, `@Min`, `@Max`
`spring-boot-starter-mail`	Envoi d'emails (confirmations, alertes)
`spring-boot-starter-actuator`	Monitoring de l'application (health, metrics, info)
`spring-boot-devtools`	Rechargement automatique en développement
`spring-boot-starter-test`	Tests unitaires et d'intégration (JUnit 5, Mockito, MockMvc)

Principes fondamentaux de Spring

— **Inversion of Control (IoC) (Inversion de Contrôle)** : le conteneur Spring gère le cycle de vie des objets (beans). Les dépendances sont injectées automatiquement, réduisant le couplage.

— **Dependency Injection (DI) (Injection de Dépendances)** : mécanisme central de Spring — les composants déclarent leurs dépendances via `@Autowired`, le conteneur les résout.

— **Auto-configuration** : Spring Boot configure automatiquement les composants détectés dans le classpath (Tomcat, Hibernate, Jackson).
— **Convention over Configuration** : réduction du boilerplate grâce aux valeurs par défaut sensées.
— **Profils** : configuration par environnement (`application-dev.properties`, `application-prod.prop`

Configuration clé

```
# Serveur
server.port=8080
server.servlet.context-path=/api

# Oracle Database
spring.datasource.url=jdbc:oracle:thin:@//192.168.1.10:1521/
   microfindb
spring.datasource.username=microfinance_app
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver

# JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.OracleDialect
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true

# Jackson (JSON)
spring.jackson.serialization.write-dates-as-timestamps=false
spring.jackson.date-format=yyyy-MM-dd'T'HH:mm:ss

# Actuator
management.endpoints.web.exposure.include=health,info,metrics
```

Listing 5.2 – application.properties — Configuration principale

5.2.2 Hibernate / JPA

TABLE 5.10 – Fiche technique — Hibernate / JPA

Propriété	Valeur
Nature	Framework ORM (Object-Relational Mapping)
Standard	JPA 3.1 (Jakarta Persistence)
Implémentation	Hibernate 6.x
Rôle	Mapping automatique entre classes Java et tables Oracle

Fonctionnalités exploitées

- **Annotations de mapping** : `@Entity`, `@Table`, `@Column`, `@Id`, `@GeneratedValue`.
- **Relations** : `@OneToMany`, `@ManyToOne`, `@OneToOne`, `@ManyToMany` avec stratégies de chargement (LAZY/EAGER).
- **Héritage** : `@Inheritance` pour les hiérarchies de classes (types de comptes).
- **JPQL** : Java Persistence Query Language pour les requêtes orientées objet.
- **Criteria API** : construction programmatique de requêtes dynamiques.
- **Cache de second niveau** : amélioration des performances pour les données peu volatiles.
- **Gestion des transactions** : via `@Transactional` de Spring.

```java
@Entity
@Table(name = "CLIENTS")
public class Client {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE,
        generator = "client_seq")
    @SequenceGenerator(name = "client_seq",
        sequenceName = "SEQ_CLIENT",
        allocationSize = 1)
    private Long id;

    @Column(name = "NOM", nullable = false, length = 100)
    private String nom;

    @Column(name = "PRENOM", nullable = false, length = 100)
    private String prenom;

    @Column(name = "NUM_PIECE", unique = true, length = 30)
    private String numPiece;
```

```
21
22      @Enumerated(EnumType.STRING)
23      @Column(name = "STATUT", length = 20)
24      private StatutClient statut;
25
26      @OneToMany(mappedBy = "client",
27                 cascade = CascadeType.ALL,
28                 fetch = FetchType.LAZY)
29      private List<Compte> comptes;
30
31      @Column(name = "DATE_CREATION")
32      private LocalDateTime dateCreation;
33
34      // Constructeurs, getters, setters ...
35  }
```

Listing 5.3 – Exemple d'entité JPA

5.2.3 Bootstrap 5

Table 5.11 – Fiche technique — Bootstrap

Propriété	Valeur
Nature	Framework CSS front-end
Version	Bootstrap 5.3+
Éditeur	Twitter / Open Source
Rôle	Composants UI, grille responsive, thèmes

Composants utilisés

- **Grille (Grid System)** : système de 12 colonnes responsive (`col-sm`, `col-md`, `col-lg`).
- **Navbar** : barre de navigation responsive avec menu déroulant.
- **Cards** : cartes pour les tableaux de bord et les fiches clients.
- **Forms** : composants de formulaire stylisés et validés.
- **Tables** : tableaux striés, hover, responsive.
- **Modals** : fenêtres modales pour les confirmations et alertes.
- **Alerts et Toasts** : notifications et messages de feedback.
- **Badges et Buttons** : étiquettes et boutons avec variantes de couleur.
- **Pagination** : navigation paginée pour les listes longues.

5.2.4 Autres bibliothèques

Table 5.12 – Bibliothèques complémentaires

Bibliothèque	Langage	Rôle
`Lombok`	Java	Génération automatique de getters, setters, constructeurs, builders
`MapStruct`	Java	Mapping automatique entre entités et DTOs
`JJWT`	Java	Création et validation de tokens JWT pour l'authentification
`Jackson`	Java	Sérialisation/désérialisation JSON
`JUnit 5`	Java	Framework de tests unitaires
`Mockito`	Java	Framework de mocking pour les tests
`Chart.js`	JavaScript	Graphiques et diagrammes pour les tableaux de bord
`DataTables`	JavaScript	Tableaux interactifs avec tri, recherche, pagination
`SweetAlert2`	JavaScript	Alertes et confirmations élégantes
`Font Awesome`	CSS	Icônes vectorielles

5.3 Système de Gestion de Base de Données

5.3.1 Oracle Database

Table 5.13 – Fiche technique — Oracle Database

Propriété	Valeur
Nature	SGBD relationnel d'entreprise
Éditeur	Oracle Corporation
Version recommandée	Oracle Database 21c XE (Express Edition) ou 19c
Modèle	Relationnel avec extensions objet
Licence	Propriétaire (XE gratuit jusqu'à 12 Go RAM, 2 CPU)
Port par défaut	1521 (listener TNS)
Langage procédural	PL/SQL
Outil d'administration	SQL*Plus, SQL Developer, Enterprise Manager

Fonctionnalités Oracle exploitées

— **Transactions ACID** : garantie d'atomicité, cohérence, isolation et durabilité — critique pour les opérations financières.
— **Séquences** : génération automatique d'identifiants uniques (`CREATE SEQUENCE`).
— **Tablespaces** : organisation physique du stockage des données.

— **Indexes** : B-tree, bitmap pour l'optimisation des requêtes de recherche.
— **Constraints** : `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`, `NOT NULL`.
— **Vues** : vues matérialisées pour les rapports et tableaux de bord.
— **Triggers** : déclencheurs `BEFORE/AFTER INSERT/UPDATE/DELETE` pour l'audit et les contrôles.
— **Procédures stockées PL/SQL** : traitements batch, calculs financiers.
— **Rôles et privilèges** : gestion fine des droits d'accès à la base.
— **Oracle Net** : configuration réseau du listener pour les connexions distantes.
— **RMAN** : sauvegarde et restauration automatisées.
— **Data Pump** : export/import de données (`expdp`, `impdp`).
— **AWR/ASH** : rapports de performance automatiques.
— **Flashback** : récupération de données après suppression accidentelle.

Pilote de connexion

Table 5.14 – Connecteurs Oracle

Connecteur	Technologie	Utilisation
`ojdbc11.jar`	Java (JDBC)	Connexion Spring Boot → Oracle
`cx_Oracle`	Python	Connexion scripts Python → Oracle
`oci8 / PDO_OCI`	PHP	Connexion PHP → Oracle (si nécessaire)

5.4 Outils de Développement et de Build

Table 5.15 – Outils de développement

Outil	Catégorie	Description
`Apache Maven`	Build	Gestion des dépendances Java, cycle de vie du build (compile → test → package → deploy)
`IntelliJ IDEA`	IDE Java	Environnement de développement intégré pour Java/Spring
`VS Code`	IDE multi	Éditeur pour PHP, HTML, CSS, JavaScript, Python
`SQL Developer`	IDE SQL	Administration Oracle, exécution de requêtes, debug PL/SQL
`Git`	Versioning	Gestion de version du code source
`GitHub / GitLab`	Forge	Hébergement du dépôt, CI/CD, issues
`Postman`	API Testing	Test des endpoints REST, documentation API
`Swagger / OpenAPI`	Documentation	Documentation automatique de l'API REST
`Docker`	Conteneurisation	Conteneurisation optionnelle de l'application et d'Oracle XE
`Apache (httpd)`	Serveur web	Hébergement des fichiers PHP/HTML

5.5 Outils d'Infrastructure et Réseau

Table 5.16 – Outils d'infrastructure

Outil	Description
`ISC DHCP Server` ou `Windows Server DHCP`	Serveur DHCP pour l'attribution dynamique des adresses IP aux machines clientes
`Oracle Linux` ou `Ubuntu Server`	Système d'exploitation serveur pour l'hébergement de l'application et de la BD
`Wireshark`	Analyse du trafic réseau pour le diagnostic
`SSH (OpenSSH)`	Administration distante des serveurs
`iptables / firewalld`	Pare-feu logiciel pour la sécurisation des serveurs
`Nagios / Zabbix`	Monitoring de l'infrastructure (optionnel)

Chapitre 6

Modélisation UML

La modélisation UML (Unified Modeling Language) permet de formaliser les différents aspects du système avant l'implémentation. Conformément au cahier des charges, six types de diagrammes sont produits.

6.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation identifie les acteurs du système et les fonctionnalités auxquelles ils ont accès.

6.1.1 Identification des acteurs

Table 6.1 – Acteurs du système

Acteur	Type	Description
Client	Externe	Personne physique ou morale bénéficiant des services de la microfinance
Caissier	Interne	Agent au guichet effectuant les opérations courantes (versements, retraits)
Gestionnaire	Interne	Responsable de la gestion des clients et des comptes, validation des opérations
Gérant / Directeur	Interne	Responsable de la validation des opérations exceptionnelles et du suivi stratégique
Administrateur BD	Interne	Responsable de la maintenance, de la sauvegarde et de la sécurité de la base Oracle
Administrateur Système	Interne	Responsable de l'infrastructure réseau, des serveurs et de la configuration DHCP
Système de notification	Système	Service automatisé d'envoi de SMS et d'emails
Système BCEAO	Externe	Réception des rapports réglementaires
Réseau VISA	Externe	Traitement des transactions par carte

6.1.2 Cas d'utilisation principaux

Table 6.2 – Cas d'utilisation par module

Module	Cas d'utilisation	Acteur(s)
Clients	CU01 — Enregistrer un client	Gestionnaire
	CU02 — Modifier les infos client	Gestionnaire
	CU03 — Rechercher un client	Caissier, Gestionnaire
	CU04 — Consulter la fiche client	Caissier, Gestionnaire
	CU05 — Supprimer / Archiver un client	Gérant
Comptes	CU06 — Ouvrir un compte	Gestionnaire
	CU07 — Consulter le solde	Caissier, Client (E-Banking)
	CU08 — Bloquer un compte	Gérant
	CU09 — Débloquer un compte	Gérant
	CU10 — Clôturer un compte	Gérant
Opérations	CU11 — Effectuer un versement	Caissier
	CU12 — Effectuer un retrait	Caissier
	CU13 — Effectuer un virement	Caissier, Client (E-Banking)
	CU14 — Consulter l'historique	Caissier, Gestionnaire
	CU15 — Surveiller les transactions	Gérant
Finances	CU16 — Calculer les agios	Système (automatique)
	CU17 — Générer les rapports BCEAO	Gérant
	CU18 — Gérer les cartes VISA	Gestionnaire
	CU19 — Envoyer un SMS Banking	Système (automatique)
Administration	CU20 — Gérer les utilisateurs	Admin Système
	CU21 — Sauvegarder la BD	Admin BD
	CU22 — Configurer le réseau	Admin Système

6.1.3 Description détaillée d'un cas d'utilisation

CU12 — Effectuer un retrait

Acteur principal

Caissier

Préconditions

Le caissier est authentifié. Le client est identifié.

Scénario nominal

1. Le caissier saisit le numéro de compte ou recherche le client.
2. Le système affiche les informations du compte (solde, statut).
3. Le caissier saisit le montant du retrait.
4. Le système vérifie : statut du compte (ACTIF), solde suffisant, montant dans les limites.
5. Le système débite le compte du montant.

6. Le système enregistre l'opération dans le journal.
7. Le système génère un reçu.
8. Le système envoie un SMS de confirmation au client.

Scénarios alternatifs — **SA1** : Compte bloqué → le système refuse l'opération et affiche un message d'erreur.
— **SA2** : Solde insuffisant → le système refuse et affiche le solde disponible.
— **SA3** : Montant supérieur au plafond → nécessite validation du gérant.

Postconditions
Le solde est mis à jour. L'opération est journalisée. Le client est notifié.

6.2 Diagramme de Classes

6.2.1 Entités principales

Table 6.3 – Classes principales du modèle de domaine

Classe	Attributs clés	Méthodes clés
`Client`	id, nom, prenom, dateNaissance, numPiece, typePiece, telephone, email, adresse, photo, statut, dateCreation	getComptes(), getHistorique(), estActif()
`Compte`	id, numCompte, typeCompte, solde, statut, dateOuverture, dateFermeture, soldeMinimal	crediter(), debiter(), bloquer(), debloquer(), cloturer()
`Operation`	id, typeOperation, montant, dateOperation, libelle, compteSource, compteDestination	valider(), annuler(), genererRecu()
`Utilisateur`	id, login, motDePasse, role, nom, prenom, dateCreation, dernierAcces	authentifier(), changerMotDePasse()
`Agios`	id, compte, montant, taux, dateDebut, dateFin, dateCalcul	calculer(), appliquer()
`CarteVISA`	id, numCarte, dateExpiration, statut, client	activer(), desactiver(), renouveler()
`Notification`	id, type, destinataire, message, dateEnvoi, statut	envoyer(), reessayer()
`JournalAudit`	id, utilisateur, action, dateAction, adresseIP, details	enregistrer()

6.2.2 Relations entre classes

Table 6.4 – Relations du modèle de domaine

Classe A	Relation	Classe B	Cardinalité
Client	possède	Compte	1..* (un client a au moins un compte)
Compte	subit	Operation	0..* (un compte peut avoir zéro ou N opérations)
Client	possède	CarteVISA	0..*
Compte	génère	Agios	0..*
Utilisateur	effectue	JournalAudit	0..*
Operation	déclenche	Notification	0..*
Utilisateur	valide	Operation	0..* (pour les opérations exceptionnelles)

6.3 Diagramme d'Activités

Le diagramme d'activités modélise les flux de travail (workflows) des processus métier clés.

6.3.1 Activité : Processus de virement

1. **Début** → Saisie des informations de virement (source, destination, montant)
2. **Décision** : Compte source existe ?
   — Non → Afficher erreur → Fin
   — Oui → suite
3. **Décision** : Compte destination existe ?
   — Non → Afficher erreur → Fin
   — Oui → suite
4. **Décision** : Compte source actif et non bloqué ?
   — Non → Afficher erreur → Fin
   — Oui → suite
5. **Décision** : Solde source $\geq$ montant ?
   — Non → Afficher "Solde insuffisant" → Fin
   — Oui → suite
6. **Décision** : Montant $\leq$ plafond virement ?
   — Non → Demande validation gérant → Validation ou Refus
   — Oui → suite
7. **Action parallèle** :
   — Débiter compte source
   — Créditer compte destination

8. Enregistrer l'opération dans le journal
9. Envoyer notifications (SMS source + SMS destination)
10. Générer les reçus
11. **Fin**

6.4 Diagramme de Séquence

Le diagramme de séquence montre l'ordre chronologique des messages échangés entre les objets pour un scénario donné.

6.4.1 Séquence : Retrait d'espèces

Table 6.5 – Séquence de messages — Retrait d'espèces

N°	Émetteur	Récepteur	Message
1	Caissier	Interface	saisirNumCompte(numCompte)
2	Interface	CompteController	GET /api/comptes/{numCompte}
3	CompteController	CompteService	getCompteByNumero(numCompte)
4	CompteService	CompteRepository	findByNumCompte(numCompte)
5	CompteRepository	Oracle DB	SELECT * FROM COMPTES WHERE NUM_COMPTE = ?
6	Oracle DB	CompteRepository	ResultSet (données du compte)
7	CompteRepository	CompteService	Compte (entité)
8	CompteService	CompteController	CompteDTO
9	CompteController	Interface	JSON (infos du compte)
10	Interface	Caissier	Affichage du solde et statut
11	Caissier	Interface	saisirMontantRetrait(montant)
12	Interface	OperationController	POST /api/operations/retrait
13	OperationController	OperationService	effectuerRetrait(compteId, montant)
14	OperationService	OperationService	vérifier statut, solde, plafond
15	OperationService	CompteRepository	save(compte) [solde mis à jour]
16	OperationService	OperationRepository	save(operation)
17	OperationService	NotificationService	envoyerSMS(client, message)
18	OperationService	OperationController	OperationDTO (confirmation)
19	OperationController	Interface	JSON (reçu)
20	Interface	Caissier	Affichage du reçu

6.5 Diagramme d'États

Le diagramme d'états modélise les différents états qu'une entité peut prendre au cours de son cycle de vie.

6.5.1 États d'un Compte

Table 6.6 – Transitions d'état d'un compte

État source	**Événement**	**État cible**	**Condition/Action**
[Initial]	Ouverture	ACTIF	Versement initial $\geq$ montant minimum
ACTIF	Bloquer	BLOQUÉ	Décision du gérant ou détection anomalie
BLOQUÉ	Débloquer	ACTIF	Autorisation du gérant
ACTIF	Clôturer	FERMÉ	Solde = 0, demande du client
BLOQUÉ	Clôturer	FERMÉ	Après régularisation
ACTIF	Découvert	EN_DÉCOUVERT	Solde < 0 (si autorisé)
EN_DÉCOUVERT	Renflouer	ACTIF	Versement ramenant le solde ≥ 0

6.5.2 États d'un Client

Table 6.7 – Transitions d'état d'un client

État source	**Événement**	**État cible**	**Condition**
[Initial]	Inscription	ACTIF	KYC validé, pièces fournies
ACTIF	Suspension	SUSPENDU	Non-conformité, inactivité prolongée
SUSPENDU	Réactivation	ACTIF	Régularisation effectuée
ACTIF	Archivage	ARCHIVÉ	Tous comptes clôturés, demande client
SUSPENDU	Archivage	ARCHIVÉ	Délai de suspension dépassé

6.6 Diagramme de Composants et de Déploiement

6.6.1 Composants logiciels

Table 6.8 – Composants du système

Composant	Technologie	Interfaces fournies / requises
Front-End Web	PHP/HTML/CSS/JS	Fournit : UI utilisateur. Requiert : API REST
API REST	Spring Boot	Fournit : endpoints REST (JSON). Requiert : Service Layer
Service Layer	Spring (Service)	Fournit : Logique métier. Requiert : Repository
Data Access	Spring Data JPA	Fournit : CRUD abstrait. Requiert : Oracle JDBC
Base de données	Oracle Database	Fournit : Stockage persistant, PL/SQL
Notification	Spring Mail + SMS API	Fournit : Envoi de notifications
Sécurité	Spring Security + JWT	Fournit : Auth, Authz, Filtres

6.6.2 Diagramme de déploiement

Table 6.9 – Nœuds de déploiement

Nœud	Adresse IP	Composants hébergés
Serveur Applicatif	IP fixe #1 (ex : 192.168.1.10)	Apache HTTP (PHP), Tomcat embarqué (Spring Boot)
Serveur BD Oracle	IP fixe #2 (ex : 192.168.1.11)	Oracle Database, Oracle Listener, RMAN
Serveur DHCP	IP fixe #3 (ex : 192.168.1.1)	Service DHCP (ISC ou Windows Server)
Machine Client 1	IP dynamique (DHCP)	Navigateur web (Chrome, Firefox)
Machine Client 2	IP dynamique (DHCP)	Navigateur web (Chrome, Firefox)

Chapitre 7

Conception de la Base de Données Oracle

7.1 Modèle Conceptuel de Données (MCD)

Le modèle conceptuel identifie les entités, leurs attributs et les associations entre elles, indépendamment de toute considération technique.

7.1.1 Entités et attributs

Table 7.1 – Entité CLIENT

Attribut	Type Oracle	Contrainte	Description
id_client	NUMBER(10)	PK, NOT NULL	Identifiant unique (séquence)
nom	VARCHAR2(100)	NOT NULL	Nom de famille
prenom	VARCHAR2(100)	NOT NULL	Prénom(s)
date_naissance	DATE	NOT NULL	Date de naissance
sexe	CHAR(1)	CHECK (M/F)	Genre
nationalite	VARCHAR2(50)		Nationalité
num_piece	VARCHAR2(30)	UNIQUE, NOT NULL	Numéro de pièce d'identité
type_piece	VARCHAR2(20)	NOT NULL	CNI, PASSEPORT, PERMIS
date_delivrance	DATE		Date de délivrance de la pièce
lieu_delivrance	VARCHAR2(100)		Lieu de délivrance
date_expiration	DATE		Date d'expiration de la pièce
telephone	VARCHAR2(20)	NOT NULL	Numéro de téléphone principal
telephone2	VARCHAR2(20)		Numéro secondaire
email	VARCHAR2(150)		Adresse email
adresse	VARCHAR2(255)		Adresse postale
ville	VARCHAR2(100)		Ville
profession	VARCHAR2(100)		Profession
revenu_mensuel	NUMBER(15,2)		Revenu mensuel déclaré
photo	BLOB		Photo d'identité

Table 7.2 – Entité COMPTE

Attribut	Type Oracle	Contrainte	Description
id_compte	NUMBER(10)	PK	Identifiant unique
num_compte	VARCHAR2(20)	UNIQUE, NOT NULL	Numéro de compte (format IMF)
id_client	NUMBER(10)	FK → CLIENT	Référence au client propriétaire
type_compte	VARCHAR2(30)	NOT NULL	EPARGNE, COURANT, DAT
solde	NUMBER(15,2)	DEFAULT 0	Solde courant
solde_minimal	NUMBER(15,2)	DEFAULT 0	Solde minimum autorisé
plafond_retrait	NUMBER(15,2)		Plafond de retrait quotidien
plafond_virement	NUMBER(15,2)		Plafond de virement quotidien
taux_interet	NUMBER(5,4)		Taux d'intérêt annuel
statut	VARCHAR2(20)	DEFAULT 'ACTIF'	ACTIF, BLOQUE, FERME
date_ouverture	TIMESTAMP	DEFAULT SYSTIMESTAMP	Date d'ouverture
date_fermeture	TIMESTAMP		Date de clôture
motif_blocage	VARCHAR2(255)		Raison du blocage
ouvert_par	VARCHAR2(50)		Utilisateur ayant ouvert

Table 7.3 – Entité OPERATION

Attribut	Type Oracle	Contrainte	Description
id_operation	NUMBER(15)	PK	Identifiant unique
type_operation	VARCHAR2(30)	NOT NULL	VERSEMENT, RETRAIT, VIREMENT
montant	NUMBER(15,2)	NOT NULL, CHECK > 0	Montant de l'opération
date_operation	TIMESTAMP	DEFAULT SYSTIMESTAMP	Date et heure
libelle	VARCHAR2(255)		Libellé de l'opération
id_compte_source	NUMBER(10)	FK → COMPTE	Compte débité
id_compte_dest	NUMBER(10)	FK → COMPTE	Compte crédité (virements)
solde_avant	NUMBER(15,2)		Solde avant l'opération
solde_apres	NUMBER(15,2)		Solde après l'opération
statut	VARCHAR2(20)	DEFAULT 'VALIDEE'	VALIDEE, ANNULEE, EN_ATTENTE
reference	VARCHAR2(30)	UNIQUE	Référence unique de l'opération
effectue_par	VARCHAR2(50)	NOT NULL	Utilisateur ayant effectué
valide_par	VARCHAR2(50)		Superviseur ayant validé

Table 7.4 – Entité UTILISATEUR

Attribut	Type Oracle	Contrainte	Description
id_utilisateur	NUMBER(10)	PK	Identifiant unique
login	VARCHAR2(50)	UNIQUE, NOT NULL	Identifiant de connexion
mot_de_passe	VARCHAR2(255)	NOT NULL	Hash du mot de passe (BCrypt)
role	VARCHAR2(30)	NOT NULL	ADMIN, GERANT, GES-TION-NAIRE, CAISSIER
nom	VARCHAR2(100)	NOT NULL	Nom
prenom	VARCHAR2(100)	NOT NULL	Prénom
email	VARCHAR2(150)	UNIQUE	Email pro-fessionnel
actif	NUMBER(1)	DEFAULT 1	1 = actif, 0 = désac-tivé
date_creation	TIMESTAMP	DEFAULT SYSTIMESTAMP	Date de création
dernier_acces	TIMESTAMP		Dernière connexion
tentatives_echouees	NUMBER(2)	DEFAULT 0	Compteur de ten-tatives échouées

Table 7.5 – Entité JOURNAL_AUDIT

Attribut	**Type Oracle**	**Contrainte**	**Description**
id_audit	NUMBER(15)	PK	Identifiant unique
id_utilisateur	NUMBER(10)	FK → UTILISATEUR	Utilisateur ayant agi
action	VARCHAR2(100)	NOT NULL	Action effectuée
table_cible	VARCHAR2(50)		Table impactée
id_enregistrement	NUMBER(15)		ID de l'enregistrement impacté
anciennes_valeurs	CLOB		Valeurs avant modification (JSON)
nouvelles_valeurs	CLOB		Valeurs après modification (JSON)
date_action	TIMESTAMP	DEFAULT SYSTIMESTAMP	Horodatage
adresse_ip	VARCHAR2(45)		Adresse IP de la machine

Table 7.6 – Entité AGIOS

Attribut	**Type Oracle**	**Contrainte**	**Description**
id_agio	NUMBER(10)	PK	Identifiant unique
id_compte	NUMBER(10)	FK → COMPTE	Compte concerné
montant_agio	NUMBER(15,2)	NOT NULL	Montant des agios calculés
taux_applique	NUMBER(5,4)	NOT NULL	Taux d'intérêt appliqué
date_debut	DATE	NOT NULL	Début de la période
date_fin	DATE	NOT NULL	Fin de la période
date_calcul	TIMESTAMP	DEFAULT SYSTIMESTAMP	Date du calcul
statut	VARCHAR2(20)	DEFAULT 'CALCULE'	CALCULE, APPLIQUE

Table 7.7 – Entité CARTE_VISA

Attribut	Type Oracle	Contrainte	Description
id_carte	NUMBER(10)	PK	Identifiant unique
num_carte	VARCHAR2(19)	UNIQUE, NOT NULL	Numéro de carte (masqué)
id_client	NUMBER(10)	FK → CLIENT	Titulaire
id_compte	NUMBER(10)	FK → COMPTE	Compte rattaché
date_expiration	DATE	NOT NULL	Date d'expiration
cvv_hash	VARCHAR2(255)	NOT NULL	Hash du CVV
plafond_quotidien	NUMBER(15,2)		Plafond quotidien de paiement
statut	VARCHAR2(20)	DEFAULT 'ACTIVE'	ACTIVE, DESACTIVEE, EXPIREE
date_creation	TIMESTAMP	DEFAULT SYSTIMESTAMP	Date de création

Table 7.8 – Entité NOTIFICATION

Attribut	Type Oracle	Contrainte	Description
id_notification	NUMBER(10)	PK	Identifiant unique
type_notif	VARCHAR2(20)	NOT NULL	SMS, EMAIL
destinataire	VARCHAR2(150)	NOT NULL	Numéro ou email
message	VARCHAR2(500)	NOT NULL	Contenu du message
date_envoi	TIMESTAMP	DEFAULT SYSTIMESTAMP	Date d'envoi
statut	VARCHAR2(20)	DEFAULT 'EN_ATTENTE'	EN_ATTENTE, ENVOYE, ECHEC
id_operation	NUMBER(15)	FK → OPERATION	Opération liée

7.2 Modèle Logique de Données (MLD)

7.2.1 Script DDL Oracle

```
-- Sequences
CREATE SEQUENCE SEQ_CLIENT START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE SEQ_COMPTE START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE SEQ_OPERATION START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE SEQ_UTILISATEUR START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE SEQ_AUDIT START WITH 1 INCREMENT BY 1;

-- Table CLIENTS
CREATE TABLE CLIENTS (
    id_client        NUMBER(10) DEFAULT SEQ_CLIENT.NEXTVAL PRIMARY
        KEY,
    nom              VARCHAR2(100) NOT NULL,
    prenom           VARCHAR2(100) NOT NULL,
    date_naissance   DATE NOT NULL,
    sexe             CHAR(1) CHECK (sexe IN ('M', 'F')),
    num_piece        VARCHAR2(30) NOT NULL UNIQUE,
    type_piece       VARCHAR2(20) NOT NULL,
    telephone        VARCHAR2(20) NOT NULL,
    email            VARCHAR2(150),
    adresse          VARCHAR2(255),
    profession       VARCHAR2(100),
```

```
21      photo            BLOB,
22      statut           VARCHAR2(20) DEFAULT 'ACTIF'
23                         CHECK (statut IN ('ACTIF','SUSPENDU','
                              ARCHIVE')),
24      date_creation    TIMESTAMP DEFAULT SYSTIMESTAMP,
25      cree_par         VARCHAR2(50)
26  );
27
28  -- Table COMPTES
29  CREATE TABLE COMPTES (
30      id_compte        NUMBER(10) DEFAULT SEQ_COMPTE.NEXTVAL PRIMARY
            KEY,
31      num_compte       VARCHAR2(20) NOT NULL UNIQUE,
32      id_client        NUMBER(10) NOT NULL
33                         REFERENCES CLIENTS(id_client),
34      type_compte      VARCHAR2(30) NOT NULL
35                         CHECK (type_compte IN
36                           ('EPARGNE','COURANT','DAT')),
37      solde            NUMBER(15,2) DEFAULT 0,
38      solde_minimal    NUMBER(15,2) DEFAULT 0,
39      plafond_retrait  NUMBER(15,2),
40      taux_interet     NUMBER(5,4),
41      statut           VARCHAR2(20) DEFAULT 'ACTIF'
42                         CHECK (statut IN
43                           ('ACTIF','BLOQUE','FERME')),
44      date_ouverture   TIMESTAMP DEFAULT SYSTIMESTAMP,
45      date_fermeture   TIMESTAMP,
46      ouvert_par       VARCHAR2(50)
47  );
48
49  -- Table OPERATIONS
50  CREATE TABLE OPERATIONS (
51      id_operation       NUMBER(15) DEFAULT SEQ_OPERATION.NEXTVAL
52                           PRIMARY KEY,
53      type_operation     VARCHAR2(30) NOT NULL
54                           CHECK (type_operation IN
55                             ('VERSEMENT','RETRAIT','VIREMENT')),
56      montant            NUMBER(15,2) NOT NULL CHECK (montant > 0),
57      date_operation     TIMESTAMP DEFAULT SYSTIMESTAMP,
58      libelle            VARCHAR2(255),
59      id_compte_source   NUMBER(10) REFERENCES COMPTES(id_compte),
```

```
    id_compte_dest     NUMBER(10) REFERENCES COMPTES(id_compte),
    solde_avant        NUMBER(15,2),
    solde_apres        NUMBER(15,2),
    statut             VARCHAR2(20) DEFAULT 'VALIDEE',
    reference          VARCHAR2(30) UNIQUE,
    effectue_par       VARCHAR2(50) NOT NULL
);

-- Index de performance
CREATE INDEX IDX_CLIENTS_STATUT ON CLIENTS(statut);
CREATE INDEX IDX_COMPTES_CLIENT ON COMPTES(id_client);
CREATE INDEX IDX_COMPTES_STATUT ON COMPTES(statut);
CREATE INDEX IDX_OPS_COMPTE_SRC ON OPERATIONS(id_compte_source);
CREATE INDEX IDX_OPS_DATE ON OPERATIONS(date_operation);
CREATE INDEX IDX_OPS_TYPE ON OPERATIONS(type_operation);
```

Listing 7.1 – Création des tables Oracle (extrait)

7.3 Procédures Stockées PL/SQL

7.3.1 Calcul des agios

```
CREATE OR REPLACE PROCEDURE CALCULER_AGIOS (
    p_id_compte IN NUMBER,
    p_date_debut IN DATE,
    p_date_fin IN DATE
) AS
    v_solde_moyen NUMBER(15,2);
    v_taux NUMBER(5,4);
    v_nb_jours NUMBER;
    v_montant_agio NUMBER(15,2);
BEGIN
    -- Recuperer le taux du compte
    SELECT taux_interet INTO v_taux
    FROM COMPTES WHERE id_compte = p_id_compte;

    -- Calculer le nombre de jours
    v_nb_jours := p_date_fin - p_date_debut;

    -- Calculer le solde moyen sur la periode
    SELECT NVL(AVG(solde_apres), 0) INTO v_solde_moyen
```

```
20      FROM OPERATIONS
21      WHERE id_compte_source = p_id_compte
22        AND date_operation BETWEEN p_date_debut AND p_date_fin;
23
24      -- Formule des agios : (Solde * Taux * Jours) / 365
25      v_montant_agio := (v_solde_moyen * v_taux * v_nb_jours) /
           365;
26
27      -- Inserer le resultat
28      INSERT INTO AGIOS (id_compte, montant_agio, taux_applique,
29                         date_debut, date_fin)
30      VALUES (p_id_compte, v_montant_agio, v_taux,
31              p_date_debut, p_date_fin);
32
33      COMMIT;
34  EXCEPTION
35      WHEN OTHERS THEN
36          ROLLBACK;
37          RAISE;
38  END;
39  /
```

Listing 7.2 – Procédure de calcul des agios

7.3.2 Trigger d'audit

```
1   CREATE OR REPLACE TRIGGER TRG_AUDIT_OPERATIONS
2   AFTER INSERT OR UPDATE OR DELETE ON OPERATIONS
3   FOR EACH ROW
4   DECLARE
5       v_action VARCHAR2(10);
6   BEGIN
7       IF INSERTING THEN
8           v_action := 'INSERT';
9       ELSIF UPDATING THEN
10          v_action := 'UPDATE';
11      ELSIF DELETING THEN
12          v_action := 'DELETE';
13      END IF;
14
15      INSERT INTO JOURNAL_AUDIT (
16          action, table_cible, id_enregistrement,
```

```
17          nouvelles_valeurs
18      ) VALUES (
19          v_action, 'OPERATIONS',
20          NVL(:NEW.id_operation, :OLD.id_operation),
21          CASE WHEN DELETING THEN NULL
22          ELSE '{"type":"' || :NEW.type_operation
23               || '","montant":' || :NEW.montant
24               || ','reference":"' || :NEW.reference || '"}'
25          END
26      );
27  END;
28  /
```

Listing 7.3 – Trigger d'audit sur les opérations

7.4 Stratégie d'Indexation

Table 7.9 – Stratégie d'indexation

Table	Colonne(s)	Type	Justification
CLIENTS	num_piece	Unique B-tree	Recherche rapide par pièce d'identité
CLIENTS	nom, prenom	B-tree composé	Recherche par nom
CLIENTS	telephone	B-tree	Recherche par téléphone
COMPTES	num_compte	Unique B-tree	Recherche par numéro de compte
COMPTES	id_client	B-tree	Jointure Client ↔ Comptes
OPERATIONS	id_compte_source	B-tree	Historique d'un compte
OPERATIONS	date_operation	B-tree	Requêtes par période
OPERATIONS	type_operation	Bitmap	Filtrage par type (peu de valeurs distinctes)
JOURNAL_AUDIT	date_action	B-tree	Recherche chronologique

7.5 Politique de Sauvegarde

Table 7.10 – Plan de sauvegarde Oracle

Type	**Fréquence**	**Détail**
Sauvegarde complète (RMAN)	Hebdomadaire	Backup full de la base et des archivelogs
Sauvegarde incrémentale	Quotidienne	Backup des blocs modifiés depuis la dernière complète
Export Data Pump	Mensuel	Export logique complet (expdp) pour archivage
Archivelog	Continu	Mode ARCHIVELOG activé pour la récupération point-in-time
Test de restauration	Trimestriel	Vérification de l'intégrité des sauvegardes

Chapitre 8

Spécification Détaillée des Fonctionnalités

8.1 Module 1 : Gestion des Clients

8.1.1 Enregistrement d'un client

Table 8.1 – Fiche fonctionnelle — Enregistrement client

Élément	Description
Identifiant	FUNC-CLI-001
Nom	Enregistrer un nouveau client
Acteur	Gestionnaire
Entrées	Nom, prénom, date de naissance, sexe, nationalité, numéro de pièce, type de pièce, téléphone, email, adresse, profession, photo
Traitements	Vérification de l'unicité de la pièce d'identité, validation des champs obligatoires, attribution d'un identifiant unique, stockage de la photo
Sorties	Fiche client créée, numéro client attribué, confirmation à l'écran
Règles	Le numéro de pièce doit être unique dans la base. Tous les champs marqués obligatoires doivent être renseignés. Le format du téléphone doit être valide.
Contraintes	Conformité KYC (Know Your Customer)

8.1.2 Mise à jour des informations

— **Champs modifiables** : téléphone, email, adresse, profession, photo, téléphone secondaire.

— **Champs non modifiables** sans autorisation : nom, prénom, date de naissance, numéro de pièce.

— **Traçabilité** : chaque modification est journalisée (ancienne valeur, nouvelle valeur, utilisateur, date).
— **Historique** : conservation de l'historique des modifications pour audit.

8.1.3 Recherche et consultation

— **Critères de recherche** : nom, prénom, numéro de pièce, numéro de téléphone, numéro de compte.
— **Recherche partielle** : recherche par fragment de nom ou de numéro (LIKE).
— **Filtres** : par statut (actif, suspendu, archivé), par type de pièce, par ville.
— **Fiche client** : affichage complet des informations personnelles, liste des comptes, historique des opérations récentes.

8.2 Module 2 : Gestion des Comptes

8.2.1 Ouverture de compte

Table 8.2 – Types de comptes et caractéristiques

Type	Solde min.	Intérêts	Plafond	Caractéristiques
Épargne	5 000 FCFA	Oui (taux fixe)	Retrait limité	Rémunération annuelle, retraits plafonnés
Courant	10 000 FCFA	Non	Selon profil	Opérations illimitées, carnet de chèques possible
DAT	50 000 FCFA	Oui (taux élevé)	Aucun retrait	Durée fixe (3, 6, 12 mois), pénalité en cas de retrait anticipé

8.2.2 Numérotation des comptes

Le numéro de compte est généré automatiquement selon le format :

`MFN-AAAA-NNNNNN`

— `MFN` : Préfixe de la microfinance
— `AAAA` : Année d'ouverture
— `NNNNNN` : Numéro séquentiel sur 6 chiffres (séquence Oracle)

8.2.3 Suivi de compte

— Consultation du solde en temps réel

— Relevé de compte sur une période
— Détail de chaque opération
— Graphique d'évolution du solde (Chart.js)
— Export en CSV/PDF

8.2.4 Blocage et déblocage

Table 8.3 – Motifs de blocage de compte

Motif	Description	Déblocage
Opération suspecte	Transaction anormale détectée par le système	Après enquête
Demande du client	Le client demande le gel temporaire du compte	Sur demande
Décision judiciaire	Saisie ou gel ordonné par une autorité	Sur décision
Inactivité	Compte inactif depuis plus de 12 mois	Sur visite client
Tentatives de fraude	Plusieurs tentatives de retrait échouées	Après vérification

8.2.5 Fermeture de compte

— **Conditions** : solde doit être exactement à zéro.
— **Processus** :
  1. Vérification qu'aucune opération n'est en cours.
  2. Retrait du solde restant si > 0.
  3. Calcul et application des intérêts/agios restants.
  4. Passage du statut à FERMÉ.
  5. Archivage des données (conservation 10 ans minimum).

8.3 Module 3 : Gestion des Opérations

8.3.1 Versement (Dépôt)

Table 8.4 – Fiche fonctionnelle — Versement

Élément	Description
Identifiant	FUNC-OPS-001
Entrées	Numéro de compte, montant, libellé (optionnel)
Validations	Compte existant et actif, montant > 0
Traitements	Crédit du compte, enregistrement de l'opération, mise à jour du solde
Contrôles	Montant $\leq$ plafond de versement quotidien (LAB), déclaration si seuil LAB atteint
Sorties	Reçu (référence, montant, nouveau solde), SMS de confirmation

8.3.2 Retrait

Table 8.5 – Fiche fonctionnelle — Retrait

Élément	Description
Identifiant	FUNC-OPS-002
Entrées	Numéro de compte, montant, identification du client
Validations	Compte actif (non bloqué), solde $\geq$ montant + solde minimal, montant $\leq$ plafond quotidien
Traitements	Débit du compte, enregistrement, mise à jour du solde
Cas d'erreur	Solde insuffisant, compte bloqué, plafond dépassé $\rightarrow$ message explicite
Sorties	Reçu, SMS de confirmation

8.3.3 Virement

Table 8.6 – Fiche fonctionnelle — Virement

Élément	Description
Identifiant	FUNC-OPS-003
Entrées	Compte source, compte destination, montant, libellé
Validations	Les deux comptes existent et sont actifs, solde source suffisant, plafond de virement respecté
Traitements	Transaction atomique : débit source + crédit destination (ACID), 2 enregistrements d'opération
Atomicité	En cas d'échec de l'une des deux opérations, annulation complète (ROLLBACK)
Sorties	Reçu pour l'émetteur, notification au destinataire

8.3.4 Surveillance des opérations

— **Alertes automatiques** :
  — Opération dépassant un seuil paramétrable.
  — Série de retraits inhabituels (fréquence ou montant).
  — Virement vers un nouveau bénéficiaire d'un montant élevé.
— **Tableau de bord des opérations** :
  — Volume quotidien par type d'opération.
  — Montant total des versements et retraits du jour.
  — Opérations en attente de validation.

8.4 Module 4 : Gestion des Finances

8.4.1 Conformité BCEAO

— **Ratios prudentiels** : calcul automatique du ratio de solvabilité, de liquidité et de couverture des risques.
— **Reporting** : génération automatique des états financiers périodiques au format BCEAO.
— **Plafonds réglementaires** : application des plafonds de taux d'intérêt fixés par la BCEAO.

8.4.2 Calcul des AGIOS

Les agios (intérêts débiteurs) sont calculés selon la formule :

$$\text{Agios} = \frac{\text{Capital} \times \text{Taux} \times \text{Nombre de jours}}{365} \tag{8.1}$$

— Calcul quotidien sur les soldes débiteurs.
— Application mensuelle ou trimestrielle.
— Conformité avec le taux d'usure BCEAO.
— Historique complet des calculs pour audit.

8.4.3 SMS Banking

Table 8.7 – Services SMS Banking

Service	Direction	Description
Notification de solde	Sortant	SMS automatique après chaque opération
Alerte de seuil	Sortant	SMS quand le solde passe sous un seuil défini
Consultation de solde	Entrant	Le client envoie un code pour recevoir son solde
Mini-relevé	Entrant	Dernières 5 opérations envoyées par SMS
Alerte de sécurité	Sortant	Notification en cas de tentative de connexion suspecte

8.4.4 E-Banking

— **Portail web client** accessible via navigateur.
— **Fonctionnalités** : consultation de solde, historique des opérations, téléchargement de relevés, virements entre comptes propres.
— **Authentification** : double authentification (mot de passe + code SMS).
— **Sessions sécurisées** : timeout automatique après inactivité.

8.4.5 Carte VISA

— **Émission** : création d'une carte liée à un compte client.
— **Activation / Désactivation** : gestion du cycle de vie de la carte.
— **Plafonds** : plafond quotidien de paiement et de retrait GAB.
— **Sécurité** : numéro masqué en base, CVV hashé, alertes SMS sur chaque transaction.
— **Interface réseau VISA** : communication avec le réseau de paiement pour le traitement des transactions.

8.4.6 Transactions avec contrôle

— **Principe des 4 yeux** : les opérations dépassant un seuil nécessitent la validation d'un second utilisateur (superviseur).

— **Workflow de validation** :

1. Le caissier saisit l'opération.
2. Le système détecte que le montant dépasse le seuil.
3. L'opération passe en statut "EN_ATTENTE".
4. Le superviseur reçoit une notification.
5. Le superviseur valide ou rejette l'opération.
6. Le système exécute ou annule l'opération.

Chapitre 9

Infrastructure Réseau

9.1 Topologie du Réseau

L'infrastructure réseau du projet repose sur un réseau local (LAN) constitué de 5 nœuds interconnectés par un commutateur (switch) Ethernet.

9.1.1 Schéma de la topologie

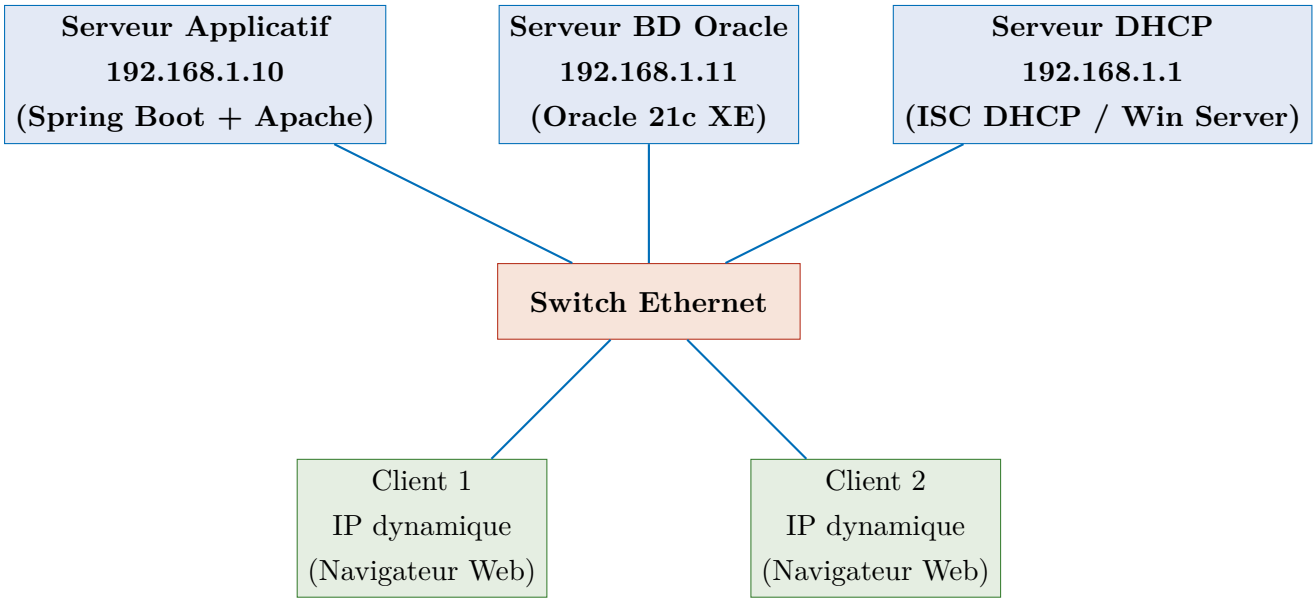

FIGURE 9.1 – Topologie réseau du projet

9.2 Plan d'Adressage IP

Table 9.1 – Plan d'adressage IP

Équipement	Adresse IP	Masque	Type	Services
Serveur Applicatif	192.168.1.10	255.255.255.0	Fixe	Apache HTTP (port 80), Tomcat (port 8080)
Serveur BD Oracle	192.168.1.11	255.255.255.0	Fixe	Oracle Listener (port 1521), EM (port 5500)
Serveur DHCP	192.168.1.1	255.255.255.0	Fixe	DHCP (ports 67-68 UDP)
Machine Client 1	192.168.1.100–200	255.255.255.0	DHCP	Navigateur HTTP/HTTPS
Machine Client 2	192.168.1.100–200	255.255.255.0	DHCP	Navigateur HTTP/HTTPS

9.2.1 Caractéristiques du réseau

— **Réseau** : 192.168.1.0/24 (classe C privée)
— **Passerelle par défaut** : 192.168.1.1 (serveur DHCP / routeur)
— **Broadcast** : 192.168.1.255
— **Plage DHCP** : 192.168.1.100 à 192.168.1.200 (101 adresses dynamiques)
— **DNS** : 8.8.8.8 / 8.8.4.4 (Google DNS) ou DNS interne
— **Durée de bail DHCP** : 24 heures

9.3 Configuration du Serveur DHCP

9.3.1 ISC DHCP Server (Linux)

```
# Parametres globaux
option domain-name "microfinance.local";
option domain-name-servers 8.8.8.8, 8.8.4.4;
default-lease-time 86400;  # 24 heures
max-lease-time 172800;     # 48 heures
authoritative;
log-facility local7;

# Definition du sous-reseau
```

```
subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.200;
    option routers 192.168.1.1;
    option subnet-mask 255.255.255.0;
    option broadcast-address 192.168.1.255;
}

# Reservations IP fixes pour les serveurs
host serveur-app {
    hardware ethernet AA:BB:CC:DD:EE:01;
    fixed-address 192.168.1.10;
}

host serveur-oracle {
    hardware ethernet AA:BB:CC:DD:EE:02;
    fixed-address 192.168.1.11;
}
```

Listing 9.1 – Configuration /etc/dhcp/dhcpd.conf

9.3.2 Windows Server DHCP (alternative)

```
# Installer le role DHCP
Install-WindowsFeature -Name DHCP -IncludeManagementTools

# Creer l'etendue
Add-DhcpServerV4Scope -Name "Microfinance LAN" `
    -StartRange 192.168.1.100 `
    -EndRange 192.168.1.200 `
    -SubnetMask 255.255.255.0 `
    -LeaseDuration 1.00:00:00

# Options d'etendue
Set-DhcpServerV4OptionValue -ScopeId 192.168.1.0 `
    -Router 192.168.1.1 `
    -DnsServer 8.8.8.8,8.8.4.4 `
    -DnsDomain "microfinance.local"

# Reservations
Add-DhcpServerV4Reservation -ScopeId 192.168.1.0 `
    -IPAddress 192.168.1.10 `
    -ClientId "AA-BB-CC-DD-EE-01" `
```

```
    -Name "Serveur-App"

Add-DhcpServerV4Reservation -ScopeId 192.168.1.0 `
    -IPAddress 192.168.1.11 `
    -ClientId "AA-BB-CC-DD-EE-02" `
    -Name "Serveur-Oracle"
```

Listing 9.2 – Configuration DHCP via PowerShell

9.4 Configuration du Serveur Applicatif

9.4.1 Apache HTTP Server

```
# /etc/httpd/conf.d/microfinance.conf (CentOS/RHEL)
# ou /etc/apache2/sites-available/microfinance.conf (Ubuntu)

<VirtualHost *:80>
    ServerName microfinance.local
    DocumentRoot /var/www/microfinance

    <Directory /var/www/microfinance>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    # Proxy vers Spring Boot (API REST)
    ProxyPreserveHost On
    ProxyPass /api http://localhost:8080/api
    ProxyPassReverse /api http://localhost:8080/api

    # Logs
    ErrorLog /var/log/httpd/microfinance-error.log
    CustomLog /var/log/httpd/microfinance-access.log combined
</VirtualHost>
```

Listing 9.3 – Configuration Apache pour le front-end PHP

9.4.2 Tomcat embarqué (Spring Boot)

Le serveur Tomcat est embarqué dans l'application Spring Boot :

— **Port** : 8080 (par défaut)
— **Contexte** : `/api`
— **Threads** : pool de threads configurable (200 par défaut)
— **Compression GZIP** : activée pour les réponses JSON

9.5 Configuration du Serveur Oracle

9.5.1 Oracle Net (Listener)

```
LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = TCP)
                 (HOST = 192.168.1.11)
                 (PORT = 1521))
    )
  )

SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (GLOBAL_DBNAME = MICROFINDB)
      (ORACLE_HOME = /opt/oracle/product/21c/dbhome_1)
      (SID_NAME = MICROFINDB)
    )
  )
```

Listing 9.4 – Configuration du listener Oracle (listener.ora)

```
MICROFINDB =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)
               (HOST = 192.168.1.11)
               (PORT = 1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = MICROFINDB)
    )
  )
```

Listing 9.5 – Configuration du client Oracle (tnsnames.ora)

9.6 Sécurité Réseau

9.6.1 Règles de pare-feu

Table 9.2 – Règles de pare-feu (iptables / firewalld)

Serveur	Port	Proto	Source	Action	Description
App	80	TCP	LAN	ACCEPT	HTTP (Apache)
App	8080	TCP	localhost	ACCEPT	Tomcat (interne uniquement)
App	22	TCP	Admin	ACCEPT	SSH administration
Oracle	1521	TCP	192.168.1.10	ACCEPT	JDBC depuis serveur app
Oracle	22	TCP	Admin	ACCEPT	SSH administration
Oracle	5500	TCP	Admin	ACCEPT	EM Express
DHCP	67-68	UDP	LAN	ACCEPT	Service DHCP
Tous	*	*	*	DROP	Règle par défaut

9.6.2 Configuration iptables

```
# Politique par defaut : tout bloquer
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# Autoriser le loopback
iptables -A INPUT -i lo -j ACCEPT

# Autoriser les connexions etablies
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Oracle Listener depuis le serveur applicatif uniquement
iptables -A INPUT -p tcp --dport 1521 \
    -s 192.168.1.10 -j ACCEPT

# SSH depuis l'admin
iptables -A INPUT -p tcp --dport 22 \
    -s 192.168.1.0/24 -j ACCEPT

# EM Express depuis l'admin
iptables -A INPUT -p tcp --dport 5500 \
    -s 192.168.1.0/24 -j ACCEPT

# Sauvegarder
```

```
25 iptables-save > /etc/sysconfig/iptables
```

Listing 9.6 – Règles iptables pour le serveur Oracle

Chapitre 10

Méthodologie de Développement

10.1 Choix de la Méthodologie

Deux méthodologies ont été envisagées pour ce projet : le **Processus Unifié (UP/RUP)** et **Scrum**. Le choix final dépend de la maturité de l'équipe et des contraintes du projet.

10.1.1 Analyse comparative

Table 10.1 – Comparaison Processus Unifié vs Scrum

Critère	Processus Unifié (UP)	Scrum
Approche	Itérative et incrémentale, pilotée par les cas d'utilisation	Itérative et incrémentale, pilotée par la valeur métier
Phases	4 phases : Inception, Élaboration, Construction, Transition	Sprints de durée fixe (2-4 semaines)
Documentation	Riche (UML, documents d'architecture)	Minimale (user stories, backlog)
Rôles	Chef de projet, architecte, analyste, développeur, testeur	Product Owner, Scrum Master, Équipe
Adaptabilité	Changements gérés entre phases	Changements à chaque sprint
Contexte académique	Très adapté (modélisation UML intégrée)	Adapté mais nécessite un PO dédié

10.1.2 Recommandation

Recommandation : Approche hybride

Pour un projet académique de 2 mois avec 5 personnes, nous recommandons une **approche hybride UP + Scrum** :

— Les **4 phases du Processus Unifié** pour structurer le cycle de vie global et guider la modélisation UML.

— Les **pratiques Scrum** (sprints, daily stand-up, backlog) pour la gestion agile du développement au quotidien.

10.2 Processus Unifié (UP) — Phases

10.2.1 Phase 1 : Inception (Semaines 1–2)

Table 10.2 – Activités de la phase Inception

Activité	Description	Livrable
Analyse du besoin	Compréhension du cahier des charges, identification des acteurs et cas d'utilisation	Document de vision
Étude de faisabilité	Évaluation technique (Spring Boot + Oracle), identification des risques	Note de faisabilité
Planification	Définition du planning, répartition des rôles, estimation des charges	Plan de projet
Modélisation initiale	Diagramme de cas d'utilisation, diagramme de classes préliminaire	Modèles UML v1
Environnement	Installation de JDK, Spring Boot, Oracle, IDE, Git	Environnement prêt

10.2.2 Phase 2 : Élaboration (Semaines 2–4)

Table 10.3 – Activités de la phase Élaboration

Activité	Description	Livrable
Architecture	Conception de l'architecture 3 tiers, design patterns	Document d'architecture
Modélisation complète	Tous les diagrammes UML (classes, séquence, activités, états, composants)	Modèles UML finaux
Conception BD	MCD, MLD, scripts DDL Oracle, procédures stockées	Schéma BD
Prototype	Maquettes d'interface, prototype fonctionnel du module client	Prototype v1
Infrastructure	Configuration des serveurs, réseau, DHCP	Infrastructure opérationnelle
Présentation intermédiaire	Préparation et soutenance intermédiaire	Support de présentation

10.2.3 Phase 3 : Construction (Semaines 4–7)

Table 10.4 – Activités de la phase Construction

Activité	Description	Livrable
Sprint 1	Module Clients (CRUD complet) + Authentification	Module Client fonctionnel
Sprint 2	Module Comptes (ouverture, suivi, blocage, clôture)	Module Comptes fonctionnel
Sprint 3	Module Opérations (versement, retrait, virement)	Module Opérations fonctionnel
Sprint 4	Module Finances (agios, rapports, SMS/E-Banking)	Module Finances fonctionnel
Données	Génération des données de test (150 clients, 600 opérations)	Base peuplée
Tests	Tests unitaires, tests d'intégration, tests fonctionnels	Rapport de tests

10.2.4 Phase 4 : Transition (Semaine 8)

Table 10.5 – Activités de la phase Transition

Activité	Description	Livrable
Déploiement	Déploiement sur les serveurs de production	Application déployée
Tests finaux	Tests de bout en bout, tests de charge, recette	Rapport de recette
Documentation	Finalisation du dossier technique, guide utilisateur	Documentation complète
Communication	Logo, messages de communication (2-3)	Supports de communication
Soutenance	Préparation et soutenance finale	Présentation finale

10.3 Pratiques Scrum Intégrées

10.3.1 Artefacts Scrum

— **Product Backlog** : liste priorisée de toutes les fonctionnalités (user stories) du projet.
— **Sprint Backlog** : sous-ensemble du Product Backlog sélectionné pour le sprint en cours.
— **Incrément** : version fonctionnelle du produit à la fin de chaque sprint.
— **Burndown Chart** : graphique de progression du sprint.

10.3.2 Cérémonies Scrum

Table 10.6 – Cérémonies Scrum adaptées

Cérémonie	Fréquence	Description
Sprint Planning	Début de sprint (semaine)	Sélection des user stories, estimation en story points
Daily Stand-up	Quotidien (15 min)	Ce que j'ai fait hier, ce que je fais aujourd'hui, obstacles
Sprint Review	Fin de sprint	Démonstration des fonctionnalités développées
Sprint Retrospective	Fin de sprint	Ce qui a bien/mal fonctionné, améliorations

10.4 Gestion de Version (Git)

10.4.1 Stratégie de branching

— **main** : branche de production, toujours stable et déployable.
— **develop** : branche d'intégration, reçoit les merges des features.
— **feature/*** : branches de fonctionnalité (ex : `feature/module-clients`).
— **hotfix/*** : corrections urgentes depuis `main`.
— **release/*** : préparation des versions de livraison.

10.4.2 Convention de commits

```
<type>(<scope>): <description>

Types :
  feat:     Nouvelle fonctionnalite
  fix:      Correction de bug
  docs:     Documentation
  style:    Formatage (pas de changement de code)
  refactor: Refactoring
  test:     Ajout ou modification de tests
  chore:    Maintenance

Exemples :
  feat(clients): ajouter le formulaire de creation
  fix(operations): corriger le calcul du solde apres retrait
  docs(readme): mettre a jour les instructions d'installation
```

Listing 10.1 – Format de message de commit

10.5 Gestion de la Qualité

10.5.1 Stratégie de test

Table 10.7 – Niveaux de test

Niveau	Outil	Description
Tests unitaires	JUnit 5 + Mockito	Test isolé de chaque service et repository
Tests d'intégration	Spring Boot Test	Test de l'interaction entre couches (Controller-Service-Repository)
Tests API	Postman + MockMvc	Test des endpoints REST (status codes, payloads)
Tests fonctionnels	Manuel	Vérification des scénarios métier de bout en bout
Tests de performance	JMeter (optionnel)	Test de charge avec simulations d'utilisateurs concurrents

10.5.2 Revue de code

— Chaque Pull Request nécessite au minimum une revue par un pair.
— Vérification de la conformité aux standards de codage.
— Vérification de la couverture de tests.
— Vérification de l'absence de vulnérabilités de sécurité.

Chapitre 11

Sécurité

11.1 Politique de Sécurité Globale

La sécurité est une préoccupation transversale dans un système de gestion financière. Elle se décline à tous les niveaux de l'architecture.

11.1.1 Principes directeurs

— **Défense en profondeur** : plusieurs couches de sécurité (réseau, application, données).
— **Moindre privilège** : chaque utilisateur et composant ne dispose que des droits strictement nécessaires.
— **Séparation des responsabilités** : aucun utilisateur ne peut réaliser seul une opération critique.
— **Audit et traçabilité** : toute action est journalisée et imputable.
— **Confidentialité des données** : chiffrement des données sensibles au repos et en transit.

11.2 Sécurité Applicative

11.2.1 Authentification

Table 11.1 – Mécanismes d'authentification

Mécanisme	Description
JWT (JSON Web Token)	Token signé contenant l'identité et les rôles, transmis dans le header `Authorization: Bearer <token>`
BCrypt	Hachage des mots de passe avec sel aléatoire, facteur de coût 12
Expiration de session	Les tokens JWT expirent après 30 minutes d'inactivité
Refresh Token	Token de rafraîchissement avec durée de vie de 7 jours
Verrouillage de compte	Après 5 tentatives échouées, le compte est verrouillé pendant 30 minutes

Flux d'authentification JWT

1. L'utilisateur envoie `POST /api/auth/login` avec login et mot de passe.
2. Le serveur vérifie les credentials via `BCrypt.matches()`.
3. Si valide : génération d'un JWT signé (HMAC-SHA256) contenant : `sub` (login), `roles`, `iat` (issued at), `exp` (expiration).
4. Le JWT est retourné au client.
5. Le client inclut le JWT dans chaque requête : `Authorization: Bearer <token>`.
6. Le filtre `JwtAuthFilter` intercepte chaque requête, valide le token et injecte l'authentification dans le `SecurityContext`.

11.2.2 Autorisation (RBAC)

Table 11.2 – Matrice des droits par rôle (RBAC)

Fonctionnalité	**Admin**	**Gérant**	**Gestionna**	**Caissier**	
Gérer les utilisateurs					
Créer un client					
Modifier un client					
Ouvrir un compte					
Bloquer / Débloquer					
Clôturer un compte					
Effectuer un versement					
Effectuer un retrait					
Effectuer un virement					
Valider opération > seuil					
Consulter les rapports					
Configurer le système					

11.2.3 Protection contre les vulnérabilités OWASP

Table 11.3 – Mesures contre les vulnérabilités OWASP Top 10

Vulnérabilité	Mesure de protection
Injection SQL	Utilisation de requêtes paramétrées via JPA/Hibernate (never concatenate)
XSS (Cross-Site Scripting)	Échappement systématique des sorties (`htmlspecialchars()` en PHP), Content-Security-Policy
CSRF	Token CSRF dans les formulaires (Spring Security CSRF protection)
Broken Authentication	JWT avec expiration courte, BCrypt, verrouillage de compte
Sensitive Data Exposure	HTTPS obligatoire, chiffrement des données sensibles en base (TDE Oracle)
Broken Access Control	Vérification du rôle à chaque endpoint (`@PreAuthorize`)
Security Misconfiguration	Headers de sécurité (X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security)
Insecure Deserialization	Validation stricte des payloads JSON, classe blanche de désérialisation
Components with Vulns	Audit régulier des dépendances Maven (OWASP Dependency Check)
Insufficient Logging	Journal d'audit complet, monitoring des événements de sécurité

11.3 Sécurité de la Base de Données

— **Comptes Oracle séparés** : un compte applicatif avec droits limités (`CONNECT`, `SELECT`, `INSERT`, `UPDATE`), un compte DBA pour l'administration.
— **Pas de `DELETE`** : les suppressions sont logiques (changement de statut à `ARCHIVE`), jamais physiques.
— **Chiffrement TDE** : les colonnes sensibles (numéro de pièce, données carte VISA) sont chiffrées au repos.
— **Audit Oracle** : activation de l'audit natif (`AUDIT SELECT ON CLIENTS BY ACCESS`).
— **Mot de passe complexe** : politique de mot de passe Oracle (longueur ≥ 12, majuscule, minuscule, chiffre, caractère spécial).
— **Connexions sécurisées** : communication JDBC via SSL/TLS.

11.4 Sécurité Réseau

— **Pare-feu** : seuls les ports nécessaires sont ouverts (cf. chapitre 9).
— **Segmentation** : le serveur Oracle n'est accessible que depuis le serveur applicatif (pas depuis les clients).
— **SSH** : accès aux serveurs exclusivement par SSH avec clé, désactivation de l'authentification par mot de passe.
— **HTTPS** : chiffrement de toutes les communications entre clients et serveur web (certificat SSL).
— **Mise à jour** : application régulière des correctifs de sécurité sur les OS et logiciels.

11.5 Conformité et Audit

11.5.1 Journal d'audit

Chaque action significative est enregistrée dans la table `JOURNAL_AUDIT` :
— Connexion / déconnexion des utilisateurs.
— Création, modification, suppression de clients et comptes.
— Toutes les opérations financières (versements, retraits, virements).
— Tentatives d'accès non autorisé.
— Modifications de configuration du système.

11.5.2 Traçabilité des opérations

— Chaque opération possède une **référence unique** non modifiable.
— Le **solde avant/après** est enregistré pour chaque transaction.
— L'**utilisateur** ayant effectué et validé l'opération est tracé.
— L'**horodatage** est précis à la milliseconde (`TIMESTAMP`).
— En cas de virement, les deux mouvements (débit/crédit) sont liés par la même référence.

Chapitre 12

Plan de Déploiement

12.1 Environnements

Table 12.1 – Environnements de déploiement

Environnement	Usage	Caractéristiques
Développement	Coding + tests unitaires	Poste développeur, BD locale, Spring Boot en mode dev
Intégration	Tests d'intégration	Serveur dédié, BD Oracle peuplée, tests de bout en bout
Production	Application finale	Serveurs dédiés (App + Oracle), réseau configuré

12.2 Procédure de Déploiement

12.2.1 Étape 1 : Préparation des serveurs

1. **Serveur applicatif** :
   — Installer le système d'exploitation (Oracle Linux, Ubuntu Server ou CentOS).
   — Installer JDK 17 : `sudo yum install java-17-openjdk-devel`.
   — Installer Apache HTTP Server : `sudo yum install httpd`.
   — Installer PHP 8.1 : `sudo yum install php php-curl php-json`.
   — Configurer le VirtualHost Apache avec le proxy vers Spring Boot.
   — Configurer le pare-feu.
2. **Serveur Oracle** :
   — Installer le système d'exploitation.
   — Installer Oracle Database 21c XE.
   — Créer la base de données `MICROFINDB`.
   — Créer les tablespaces et les utilisateurs.

— Exécuter les scripts DDL de création des tables.
— Configurer le listener Oracle Net.
— Configurer le pare-feu.
— Mettre en place RMAN pour les sauvegardes.

3. **Serveur DHCP** :
   — Installer le service DHCP (ISC ou Windows Server).
   — Configurer le fichier `dhcpd.conf` (cf. chapitre 9).
   — Démarrer le service et vérifier l'attribution des adresses.

12.2.2 Étape 2 : Déploiement du back-end

```
# Build du JAR
cd microfinance-api/
mvn clean package -DskipTests

# Copier le JAR sur le serveur
scp target/microfinance-0.1.0.jar \
    admin@192.168.1.10:/opt/microfinance/

# Sur le serveur - lancer l'application
cd /opt/microfinance/
nohup java -jar microfinance-0.1.0.jar \
    --spring.profiles.active=prod \
    > logs/app.log 2>&1 &

# Ou creer un service systemd
sudo cat > /etc/systemd/system/microfinance.service << EOF
[Unit]
Description=MicroFin Manager Application
After=network.target

[Service]
Type=simple
User=microfinance
ExecStart=/usr/bin/java -jar \
    /opt/microfinance/microfinance-0.1.0.jar \
    --spring.profiles.active=prod
Restart=on-failure
RestartSec=10

[Install]
```

```
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable microfinance
sudo systemctl start microfinance
```

Listing 12.1 – Commandes de build et déploiement Spring Boot

12.2.3 Étape 3 : Déploiement du front-end

```
# Copier les fichiers web
scp -r webapp/* admin@192.168.1.10:/var/www/microfinance/

# Configurer les permissions
sudo chown -R apache:apache /var/www/microfinance/
sudo chmod -R 755 /var/www/microfinance/

# Configurer le fichier de connexion API
cat > /var/www/microfinance/includes/config.php << 'EOF'
<?php
define('API_BASE_URL', 'http://localhost:8080/api');
define('APP_NAME', 'MicroFin Manager');
define('APP_VERSION', '1.0.0');
EOF

# Redemarrer Apache
sudo systemctl restart httpd
```

Listing 12.2 – Déploiement du front-end PHP

12.2.4 Étape 4 : Peuplement de la base

```
# Installer les dependances Python
pip install faker cx_Oracle

# Executer le script de generation
python3 generer_donnees.py \
    --clients 150 \
    --operations 600 \
    --host 192.168.1.11 \
```

```
9       --port 1521 \
10      --service MICROFINDB
11
12  # Ou importer via Data Pump
13  impdp microfinance_app/password@MICROFINDB \
14      directory=DATA_PUMP_DIR \
15      dumpfile=microfinance_data.dmp \
16      table_exists_action=append
```

Listing 12.3 – Génération et insertion des données

12.2.5 Étape 5 : Vérification post-déploiement

Table 12.2 – Checklist de vérification post-déploiement

Vérification	Commande / Action	Résultat attendu
API disponible	`curl http://192.168.1.10:8080/api/actuator/health`	`{"status":"UP"}`
Front accessible	Navigateur → `http://192.168.1.10/`	Page de login
BD connectée	Connexion SQL Developer à 192.168.1.11 :1521	Requêtes OK
DHCP actif	Connecter un client, vérifier IP obtenue	IP dans la plage
Login fonctionne	Connexion avec un compte test	Accès au tableau de bord
Opération test	Effectuer un versement test	Solde mis à jour
SMS envoyé	Vérifier le log des notifications	SMS envoyé ou journalisé

12.3 Gestion des Mises à Jour

— **Zéro downtime** : déploiement en arrêtant le service systemd, remplacement du JAR, redémarrage.
— **Rollback** : conservation de la version précédente du JAR pour retour arrière rapide.
— **Migration BD** : utilisation de scripts de migration versionnés (Flyway ou Liquibase).
— **Sauvegarde pré-déploiement** : export Data Pump avant chaque mise à jour.

Chapitre 13

Organisation de l'Équipe

13.1 Composition de l'Équipe

L'équipe projet est composée de 5 membres aux compétences complémentaires.

Table 13.1 – Composition de l'équipe projet

N°	Rôle	Profil	Responsabilités principales
1	Chef de projet / Architecte	Senior	Coordination générale, architecture technique, revue de code, communication avec les encadrants
2	Développeur Back-end	Intermédiaire	Développement des API REST Spring Boot, logique métier, services, sécurité (JWT)
3	Développeur Front-end	Intermédiaire	Interfaces PHP/HTML/CSS/JS, intégration avec l'API, ergonomie, responsive design
4	DBA / Administrateur BD	Intermédiaire	Conception Oracle, scripts DDL/DML, procédures stockées, triggers, optimisation, sauvegardes
5	Testeur / Intégrateur	Junior	Tests unitaires et fonctionnels, génération de données (Python), documentation, déploiement

13.2 Matrice RACI

La matrice RACI définit les niveaux de responsabilité pour chaque livrable du projet.

Table 13.2 – Matrice RACI

Activité	**Chef**	**Back**	**Front**	**DBA**	**Test**
Architecture technique	**R**	C	C	C	I
Modélisation UML	**R**	C	C	C	I
Conception Base de Données	A	C	I	**R**	I
API REST (Spring Boot)	A	**R**	I	C	C
Interfaces PHP	A	C	**R**	I	C
Procédures stockées PL/SQL	A	C	I	**R**	I
Tests unitaires (JUnit)	A	**R**	I	I	C
Tests fonctionnels	A	C	C	I	**R**
Génération de données	I	I	I	C	**R**
Infrastructure réseau	**R**	I	I	C	C
Sécurité	**R**	C	C	C	C
Documentation technique	**R**	C	C	C	C
Déploiement	A	C	C	C	**R**
Présentation / Soutenance	**R**	C	C	C	C

Légende RACI

R — Responsable : Celui qui exécute la tâche.

A — Approbateur : Celui qui valide le résultat (un seul par tâche).

C — Consulté : Celui dont l'avis est sollicité.

I — Informé : Celui qui est tenu au courant de l'avancement.

13.3 Outils de Collaboration

Table 13.3 – Outils de collaboration de l'équipe

Catégorie	Outil	Usage
Gestion de version	Git + GitHub/GitLab	Hébergement du code source, pull requests, revues de code
Communication	WhatsApp / Slack	Communication quotidienne, partage rapide d'informations
Gestion de tâches	Trello / GitHub Projects	Tableau Kanban (To Do, In Progress, In Review, Done)
Documentation	Google Docs / LaTeX	Rédaction collaborative du dossier technique
Visioconférence	Google Meet / Zoom	Réunions d'équipe, daily stand-ups
IDE	IntelliJ IDEA / VS Code	Développement Java (IntelliJ), PHP/HTML/CSS (VS Code)
Base de données	SQL Developer / DBeaver	Administration et interrogation de la base Oracle
Tests API	Postman	Tests des endpoints REST
Design	Figma (optionnel)	Maquettes d'interface utilisateur

13.4 Répartition du Travail par Sprint

13.4.1 Sprint 1 — Module Clients (Semaines 4–5)

Table 13.4 – Répartition Sprint 1 — Module Clients

Membre	Tâches
Chef de projet	Supervision, revue de code, architecture des endpoints clients
Back-end	`ClientController`, `ClientService`, `ClientRepository`, DTOs, validation
Front-end	Pages : liste des clients, formulaire de création, fiche client, recherche
DBA	Table CLIENTS, contraintes, triggers d'audit, index, procédure de recherche
Testeur	Tests `ClientServiceTest`, cas de test fonctionnels, données de test

13.4.2 Sprint 2 — Module Comptes (Semaines 5–6)

Table 13.5 – Répartition Sprint 2 — Module Comptes

Membre	Tâches
Chef de projet	Supervision, validation des règles métier (solde minimum, types de comptes)
Back-end	`CompteController`, `CompteService`, gestion des statuts, génération de numéros
Front-end	Pages : ouverture de compte, tableau de bord, détail du compte, blocage/clôture
DBA	Table COMPTES, historique des statuts, vue consolidée, procédures PL/SQL
Testeur	Tests `CompteServiceTest`, scénarios de blocage/déblocage, tests d'intégration

13.4.3 Sprint 3 — Module Opérations (Semaines 6–7)

Table 13.6 – Répartition Sprint 3 — Module Opérations

Membre	Tâches
Chef de projet	Supervision, validation de l'atomicité des virements, règles de double validation
Back-end	`OperationController`, `OperationService`, transactions ACID, workflow 4-eyes
Front-end	Pages : versement, retrait, virement, historique des opérations, reçu
DBA	Table OPERATIONS, triggers de mise à jour des soldes, procédure de virement, partitionnement
Testeur	Tests transactionnels, scénarios de concurrence, génération de 600 opérations (Python)

13.4.4 Sprint 4 — Module Finances (Semaine 7–8)

Table 13.7 – Répartition Sprint 4 — Module Finances

Membre	Tâches
Chef de projet	Supervision, conformité BCEAO, intégration globale
Back-end	Calcul des agios, API pour rapports, service SMS/E-Banking, intégration VISA
Front-end	Pages : tableau de bord financier, rapports, paramétrage SMS
DBA	Vues récapitulatives, procédures de calcul d'agios, rapprochement
Testeur	Tests de conformité, tests de bout en bout, préparation de la démo

13.5 Gestion des Risques

Table 13.8 – Registre des risques

Risque	Probabilité	Impact	Mitigation
Indisponibilité d'un membre	Moyenne	Fort	Formation croisée, documentation du code, pair programming
Difficultés avec Oracle	Moyenne	Fort	Prévoir Oracle XE en local, scripts de restauration rapide
Retard sur un module	Élevée	Moyen	Sprints courts, priorisation (MoSCoW), réallocation des tâches
Problèmes d'intégration API	Moyenne	Fort	Contrats d'API définis tôt, tests Postman dès le Sprint 1
Perte de données de test	Faible	Fort	Sauvegardes quotidiennes RMAN, scripts de peuplement reproductibles
Conflits Git	Élevée	Faible	Convention de branching, merges fréquents, revue de code

Chapitre 14

Livrables et Communication

14.1 Liste des Livrables

Table 14.1 – Inventaire exhaustif des livrables

N°	Livrable	Description	Format
L01	Dossier technique	Document complet décrivant architecture, technologies, BD, sécurité	PDF (LaTeX)
L02	Code source back-end	Projet Maven Spring Boot complet (contrôleurs, services, repositories)	Java (Git)
L03	Code source front-end	Application web PHP/HTML/CSS/JS	PHP (Git)
L04	Scripts BD	Scripts DDL, DML, PL/SQL (procédures, triggers, vues)	SQL (Git)
L05	Scripts de peuplement	Script Python de génération de données de test	Python (Git)
L06	Modèles UML	Diagrammes de cas d'utilisation, classes, séquence, activités, états, déploiement	PDF / PNG
L07	Guide d'installation	Procédure d'installation et de configuration de l'application	PDF
L08	Guide utilisateur	Manuel d'utilisation de l'application pour les opérateurs	PDF
L09	Rapport de tests	Résultats des tests unitaires, d'intégration et fonctionnels	PDF
L10	Présentation finale	Support de soutenance (PowerPoint / Beamer)	PPTX / PDF
L11	Logo du projet	Identité visuelle du logiciel MicroFin Manager	PNG / SVG
L12	Messages de communication	2 à 3 messages de communication du projet	PDF

14.2 Arborescence des Livrables

```
microfinance-project/
|
|-- dossier_technique/           # L01 : Dossier technique LaTeX
|   |-- main.tex
|   |-- chapters/
|       |-- 00_page_de_garde.tex ... 15_annexes.tex
|
|-- microfinance-api/            # L02 : Back-end Spring Boot
|   |-- pom.xml
|   |-- src/
|   |   |-- main/
|   |   |   |-- java/com/microfinance/
|   |   |   |   |-- config/         # Configuration Spring
|   |   |   |   |-- controller/     # Controleurs REST
|   |   |   |   |-- service/        # Services metier
|   |   |   |   |-- repository/     # Repositories JPA
|   |   |   |   |-- model/          # Entites JPA
|   |   |   |   |-- dto/            # Data Transfer Objects
|   |   |   |   |-- security/       # JWT, filtres, config
|   |   |   |   |-- exception/      # Gestion des erreurs
|   |   |   |   `-- util/           # Utilitaires
|   |   |   `-- resources/
|   |   |       |-- application.yml
|   |   |       `-- application-prod.yml
|   |   `-- test/
|   |       `-- java/com/microfinance/
|   |           |-- controller/
|   |           |-- service/
|   |           `-- repository/
|   `-- target/
|       `-- microfinance-0.1.0.jar
|
|-- webapp/                      # L03 : Front-end PHP
|   |-- index.php                # Page de connexion
|   |-- includes/
|   |   |-- config.php           # Configuration API
|   |   |-- header.php           # En-tete commun
|   |   |-- footer.php           # Pied de page commun
|   |   |-- auth.php             # Fonctions d'authentification
|   |   `-- api_client.php       # Client HTTP pour l'API
```

```
|   |-- assets/
|   |   |-- css/
|   |   |   |-- bootstrap.min.css
|   |   |   `-- style.css        # Styles personnalises
|   |   |-- js/
|   |   |   |-- bootstrap.bundle.min.js
|   |   |   |-- jquery.min.js
|   |   |   `-- app.js           # Scripts personnalises
|   |   `-- img/
|   |       `-- logo.png
|   |-- clients/
|   |   |-- index.php            # Liste des clients
|   |   |-- create.php           # Creation d'un client
|   |   |-- edit.php             # Modification
|   |   `-- view.php             # Fiche client
|   |-- comptes/
|   |   |-- index.php            # Liste des comptes
|   |   |-- create.php           # Ouverture de compte
|   |   `-- view.php             # Detail du compte
|   |-- operations/
|   |   |-- versement.php
|   |   |-- retrait.php
|   |   |-- virement.php
|   |   `-- historique.php
|   |-- finances/
|   |   |-- dashboard.php
|   |   |-- agios.php
|   |   `-- rapports.php
|   `-- admin/
|       |-- users.php
|       `-- settings.php
|
|-- database/                    # L04 : Scripts BD
|   |-- ddl/
|   |   |-- 01_create_tablespace.sql
|   |   |-- 02_create_users.sql
|   |   |-- 03_create_tables.sql
|   |   |-- 04_create_indexes.sql
|   |   `-- 05_create_constraints.sql
|   |-- plsql/
|   |   |-- pkg_clients.sql
```

```
|   |   |-- pkg_comptes.sql
|   |   |-- pkg_operations.sql
|   |   |-- trg_audit.sql
|   |   `-- trg_solde.sql
|   |-- views/
|   |   |-- vw_clients_comptes.sql
|   |   |-- vw_operations_jour.sql
|   |   `-- vw_soldes_consolides.sql
|   `-- data/
|       |-- data_roles.sql
|       |-- data_utilisateurs.sql
|       `-- data_parametres.sql
|
|-- scripts/                    # L05 : Scripts de peuplement
|   |-- generer_donnees.py
|   |-- requirements.txt
|   `-- README.md
|
|-- docs/                       # Documentation supplementaire
|   |-- uml/                    # L06 : Modeles UML
|   |   |-- use_case.png
|   |   |-- class_diagram.png
|   |   |-- sequence_*.png
|   |   `-- deployment.png
|   |-- guide_installation.pdf  # L07
|   |-- guide_utilisateur.pdf   # L08
|   `-- rapport_tests.pdf       # L09
|
|-- presentation/               # L10 : Soutenance
|   `-- soutenance_finale.pptx
|
|-- communication/              # L11, L12
|   |-- logo/
|   |   |-- logo_microfinmanager.png
|   |   `-- logo_microfinmanager.svg
|   `-- messages/
|       |-- com_message_1.pdf
|       |-- com_message_2.pdf
|       `-- com_message_3.pdf
|
`-- README.md
```

Listing 14.1 – Arborescence complète du projet

14.3 Communication du Projet

14.3.1 Logo

Le logo du projet « MicroFin Manager » doit respecter les critères suivants :

— **Symbolique** : évoquer la microfinance, la confiance, la croissance.
— **Couleurs** : palette professionnelle (bleu, vert, doré) évoquant la stabilité et la prospérité.
— **Lisibilité** : lisible en petit format (favicon) comme en grand (affiche).
— **Formats** : PNG (fond transparent) et SVG (vectoriel).

14.3.2 Messages de communication

Trois messages de communication doivent être produits :

Table 14.2 – Messages de communication

N°	Type	Contenu
1	Affiche de présentation	Présentation du logiciel, fonctionnalités clés, identité visuelle
2	Flyer technique	Architecture technique, technologies utilisées, schéma d'infrastructure
3	Post réseaux sociaux	Message court et percutant pour annoncer le projet, avec visuel

14.4 Guide Utilisateur — Sommaire

Le guide utilisateur destiné aux opérateurs de la microfinance comprendra les sections suivantes :

1. **Connexion au système** : URL d'accès, identifiants, première connexion, changement de mot de passe.
2. **Tableau de bord** : description des indicateurs, navigation.
3. **Gestion des clients** : création, modification, recherche, consultation de l'historique.
4. **Gestion des comptes** : ouverture, consultation du solde, blocage, déblocage, clôture.
5. **Opérations courantes** : versement, retrait, virement interne, impression de reçu.
6. **Opérations spéciales** : validation double (4-eyes), opérations au-delà du seuil.

7. **Rapports et états** : consultation des rapports journaliers, relevés de compte.
8. **Dépannage** : problèmes fréquents et résolutions.

Chapitre 15

Annexes

15.1 Annexe A — Configuration Spring Boot complète

```yaml
spring:
  application:
    name: microfinance-api

  datasource:
    url: jdbc:oracle:thin:@192.168.1.11:1521:MICROFINDB
    username: microfinance_app
    password: ${DB_PASSWORD}
    driver-class-name: oracle.jdbc.OracleDriver
    hikari:
      connection-timeout: 30000
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 600000
      max-lifetime: 1800000
      pool-name: MicroFinHikariPool

  jpa:
    database-platform: org.hibernate.dialect.Oracle12cDialect
    hibernate:
      ddl-auto: validate
    show-sql: false
    properties:
      hibernate:
        format_sql: true
```

```yaml
        use_sql_comments: true
        default_batch_fetch_size: 25

  jackson:
    date-format: yyyy-MM-dd HH:mm:ss
    time-zone: Africa/Abidjan
    default-property-inclusion: non_null

server:
  port: 8080
  servlet:
    context-path: /api
  compression:
    enabled: true
    mime-types: application/json,text/html,text/css
    min-response-size: 1024

app:
  jwt:
    secret: ${JWT_SECRET}
    expiration-ms: 1800000       # 30 minutes
    refresh-expiration-ms: 604800000  # 7 jours
  security:
    max-login-attempts: 5
    lock-duration-minutes: 30
  operations:
    seuil-validation-double: 1000000  # 1 000 000 FCFA
    solde-minimum: 5000               # 5 000 FCFA

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
  endpoint:
    health:
      show-details: when_authorized
```

Listing 15.1 – application.yml (profil par défaut)

```yaml
spring:
  datasource:
```

```yaml
    url: jdbc:oracle:thin:@192.168.1.11:1521:MICROFINDB
    username: microfinance_app
    password: ${DB_PASSWORD_PROD}
    hikari:
      maximum-pool-size: 30

  jpa:
    show-sql: false

server:
  port: 8080

logging:
  level:
    root: WARN
    com.microfinance: INFO
    org.springframework.security: WARN
  file:
    name: /var/log/microfinance/application.log
    max-size: 50MB
    max-history: 30
```

Listing 15.2 – application-prod.yml (profil production)

15.2 Annexe B — Script Python de génération de données

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Generateur de donnees de test pour MicroFin Manager.
Produit environ 150 clients et 600 operations.
"""

import cx_Oracle
import random
from faker import Faker
from datetime import datetime, timedelta

fake = Faker('fr_FR')
```

```python

# Configuration
DB_CONFIG = {
    'user': 'microfinance_app',
    'password': 'ChangeMe123!',
    'dsn': cx_Oracle.makedsn('192.168.1.11', 1521,
                             service_name='MICROFINDB')
}

TYPES_PIECES = ['CNI', 'PASSEPORT', 'PERMIS', 'CARTE_CONSULAIRE']
TYPES_COMPTES = ['EPARGNE', 'COURANT']
TYPES_OPERATIONS = ['VERSEMENT', 'RETRAIT', 'VIREMENT']
STATUTS_COMPTES = ['ACTIF']

def generer_reference():
    """Genere une reference unique d'operation."""
    return f"OP{datetime.now():%Y%m%d}" \
           f"{random.randint(100000, 999999)}"

def generer_numero_compte():
    """Genere un numero de compte au format BCEAO."""
    code_pays = "CI"     # Cote d'Ivoire
    code_banque = "0142"
    code_guichet = "0001"
    num_compte = f"{random.randint(10000000, 99999999)}"
    cle_rib = f"{random.randint(10, 99)}"
    return f"{code_pays}{code_banque}{code_guichet}" \
           f"{num_compte}{cle_rib}"

def creer_clients(cursor, n=150):
    """Insert N clients dans la base."""
    clients_ids = []
    sql = """
    INSERT INTO CLIENTS
        (NOM, PRENOM, DATE_NAISSANCE, LIEU_NAISSANCE,
         SEXE, NATIONALITE, TELEPHONE, EMAIL,
         ADRESSE, TYPE_PIECE, NUMERO_PIECE,
         PROFESSION, STATUT)
    VALUES
        (:1, :2, :3, :4, :5, :6, :7, :8,
         :9, :10, :11, :12, 'ACTIF')
```

```
55      RETURNING ID_CLIENT INTO :out_id
56      """
57      for _ in range(n):
58          sexe = random.choice(['M', 'F'])
59          prenom = fake.first_name_male() if sexe == 'M' \
60                   else fake.first_name_female()
61
62          out_id = cursor.var(cx_Oracle.NUMBER)
63          cursor.execute(sql, [
64              fake.last_name().upper(),
65              prenom,
66              fake.date_of_birth(
67                  minimum_age=18, maximum_age=75),
68              fake.city(),
69              sexe,
70              'Ivoirienne',
71              f"+225{random.randint(0100000000, 0799999999)}",
72              fake.email(),
73              fake.address().replace('\n', ', '),
74              random.choice(TYPES_PIECES),
75              fake.unique.bothify('??######').upper(),
76              fake.job(),
77              out_id
78          ])
79          clients_ids.append(int(out_id.getvalue()[0]))
80
81      print(f"  -> {len(clients_ids)} clients crees.")
82      return clients_ids
83
84  def creer_comptes(cursor, clients_ids):
85      """Cree 1 ou 2 comptes par client."""
86      comptes = []
87      sql = """
88      INSERT INTO COMPTES
89          (ID_CLIENT, NUMERO_COMPTE, TYPE_COMPTE,
90           SOLDE, STATUT_COMPTE, DATE_OUVERTURE)
91      VALUES (:1, :2, :3, :4, 'ACTIF', SYSDATE)
92      RETURNING ID_COMPTE INTO :out_id
93      """
94      for cid in clients_ids:
95          nb_comptes = random.choices([1, 2],
```

```
                                        weights=[70, 30])[0]
        for _ in range(nb_comptes):
            out_id = cursor.var(cx_Oracle.NUMBER)
            solde_initial = random.randint(5000, 500000)
            cursor.execute(sql, [
                cid,
                generer_numero_compte(),
                random.choice(TYPES_COMPTES),
                solde_initial,
                out_id
            ])
            comptes.append({
                'id': int(out_id.getvalue()[0]),
                'solde': solde_initial
            })

    print(f"  -> {len(comptes)} comptes crees.")
    return comptes

def creer_operations(cursor, comptes, n=600):
    """Genere N operations aleatoires."""
    sql = """
    INSERT INTO OPERATIONS
        (ID_COMPTE, TYPE_OPERATION, MONTANT,
         REFERENCE, SOLDE_AVANT, SOLDE_APRES,
         DATE_OPERATION, STATUT)
    VALUES (:1, :2, :3, :4, :5, :6,
            SYSDATE - :7, 'VALIDEE')
    """
    ops_count = 0
    for _ in range(n):
        compte = random.choice(comptes)
        type_op = random.choice(TYPES_OPERATIONS)
        montant = random.randint(1000, 200000)

        solde_avant = compte['solde']
        if type_op == 'VERSEMENT':
            solde_apres = solde_avant + montant
        elif type_op == 'RETRAIT':
            if solde_avant - montant < 5000:
                continue  # Solde insuffisant
```

```
137                 solde_apres = solde_avant - montant
138             else:  # VIREMENT
139                 if solde_avant - montant < 5000:
140                     continue
141                 solde_apres = solde_avant - montant
142 
143             jours_passe = random.randint(0, 90)
144             cursor.execute(sql, [
145                 compte['id'], type_op, montant,
146                 generer_reference(), solde_avant,
147                 solde_apres, jours_passe
148             ])
149             compte['solde'] = solde_apres
150             ops_count += 1
151 
152     print(f"  -> {ops_count} operations creees.")
153 
154 def main():
155     """Point d'entree principal."""
156     print("Connexion a Oracle...")
157     conn = cx_Oracle.connect(**DB_CONFIG)
158     cursor = conn.cursor()
159 
160     try:
161         print("Generation des clients...")
162         clients = creer_clients(cursor, 150)
163 
164         print("Generation des comptes...")
165         comptes = creer_comptes(cursor, clients)
166 
167         print("Generation des operations...")
168         creer_operations(cursor, comptes, 600)
169 
170         conn.commit()
171         print("Donnees generees avec succes !")
172 
173     except Exception as e:
174         conn.rollback()
175         print(f"Erreur : {e}")
176         raise
177 
```

```
    finally:
        cursor.close()
        conn.close()

if __name__ == '__main__':
    main()
```

Listing 15.3 – generer_donnees.py (extrait)

15.3 Annexe C — Exemples de requêtes API (Postman)

15.3.1 Authentification

```
POST http://192.168.1.10:8080/api/auth/login
Content-Type: application/json

{
    "login": "admin",
    "password": "P@ssw0rd2024!"
}

--- Reponse 200 OK ---
{
    "token": "eyJhbGciOiJIUzI1NiIsI...",
    "refreshToken": "dGhpcyBpcyBhIHJlZnJ...",
    "type": "Bearer",
    "expiresIn": 1800000,
    "roles": ["ROLE_ADMIN"]
}
```

Listing 15.4 – POST /api/auth/login

15.3.2 Créer un client

```
POST http://192.168.1.10:8080/api/clients
Authorization: Bearer eyJhbGciOiJIUzI1NiIsI...
Content-Type: application/json

{
```

```
    "nom": "KOUASSI",
    "prenom": "Amani Jean-Luc",
    "dateNaissance": "1990-05-15",
    "lieuNaissance": "Abidjan",
    "sexe": "M",
    "nationalite": "Ivoirienne",
    "telephone": "+2250102030405",
    "email": "jl.kouassi@email.ci",
    "adresse": "Cocody, Rue des Jardins, Abidjan",
    "typePiece": "CNI",
    "numeroPiece": "CI1234567890",
    "profession": "Commercant"
}

--- Reponse 201 Created ---
{
    "idClient": 151,
    "nom": "KOUASSI",
    "prenom": "Amani Jean-Luc",
    "statut": "ACTIF",
    "dateCreation": "2024-12-15 10:30:00"
}
```

Listing 15.5 – POST /api/clients

15.3.3 Effectuer un virement

```
POST http://192.168.1.10:8080/api/operations/virements
Authorization: Bearer eyJhbGciOiJIUzI1NiIsI...
Content-Type: application/json

{
    "compteSource": 42,
    "compteDestination": 78,
    "montant": 250000,
    "motif": "Reglement facture fournisseur"
}

--- Reponse 201 Created ---
{
    "reference": "OP20241215847293",
    "typeOperation": "VIREMENT",
```

```
    "montant": 250000,
    "compteSource": {
        "idCompte": 42,
        "soldeAvant": 1500000,
        "soldeApres": 1250000
    },
    "compteDestination": {
        "idCompte": 78,
        "soldeAvant": 320000,
        "soldeApres": 570000
    },
    "dateOperation": "2024-12-15 10:45:12",
    "statut": "VALIDEE"
}
```

Listing 15.6 – POST /api/operations/virements

15.4 Annexe D — Glossaire

Terme	Définition
ACID	Atomicité, Cohérence, Isolation, Durabilité — propriétés garantissant la fiabilité des transactions.
Agios	Intérêts facturés par un établissement financier sur un découvert ou un crédit.
API	Application Programming Interface — interface permettant la communication entre composants logiciels.
BCEAO	Banque Centrale des États de l'Afrique de l'Ouest — régulateur monétaire de l'UEMOA.
BCrypt	Algorithme de hachage de mots de passe basé sur Blowfish, résistant aux attaques par force brute.
CRUD	Create, Read, Update, Delete — opérations de base sur les données.
CSRF	Cross-Site Request Forgery — attaque forçant un utilisateur authentifié à exécuter des actions non voulues.
DAO	Data Access Object — pattern de conception pour l'accès aux données.
DDL	Data Definition Language — commandes SQL de définition de structure (CREATE, ALTER, DROP).

Terme	Définition
DHCP	Dynamic Host Configuration Protocol — protocole d'attribution automatique d'adresses IP.
DML	Data Manipulation Language — commandes SQL de manipulation de données (INSERT, UPDATE, DELETE).
DTO	Data Transfer Object — objet de transfert de données entre couches applicatives.
E-Banking	Services bancaires accessibles en ligne via un navigateur ou une application mobile.
FCFA	Franc de la Communauté Financière Africaine — monnaie de la zone UEMOA.
Hibernate	Framework ORM Java permettant le mapping objet-relationnel.
JDBC	Java DataBase Connectivity — API Java pour la connexion aux bases de données.
JPA	Java Persistence API — spécification Java pour la persistance des données.
JWT	JSON Web Token — standard de token d'authentification signé.
KYC	Know Your Customer — procédure d'identification obligatoire des clients.
MCD	Modèle Conceptuel de Données — représentation abstraite de la structure des données.
MLD	Modèle Logique de Données — traduction du MCD en tables relationnelles.
MVC	Model-View-Controller — pattern architectural de séparation des responsabilités.
OWASP	Open Web Application Security Project — organisation promouvant la sécurité applicative.
PL/SQL	Procedural Language / SQL — extension procédurale d'Oracle pour le SQL.
RACI	Responsible, Accountable, Consulted, Informed — matrice de responsabilités.
REST	Representational State Transfer — style architectural pour les API web.
RIB	Relevé d'Identité Bancaire — identifiant unique d'un compte bancaire.

Terme	Définition
RMAN	Recovery Manager — outil de sauvegarde et restauration Oracle.
SMS	Short Message Service — service de messagerie courte pour les notifications.
Spring Boot	Framework Java simplifiant la création d'applications Spring autonomes.
TDE	Transparent Data Encryption — chiffrement transparent des données Oracle au repos.
UEMOA	Union Économique et Monétaire Ouest-Africaine — zone économique régionale.
UML	Unified Modeling Language — langage de modélisation standardisé.
VISA	Réseau international de paiement par carte.
XSS	Cross-Site Scripting — injection de scripts malveillants dans des pages web.

15.5 Annexe E — Bibliographie et Références

1. **Spring Boot Documentation** — https://docs.spring.io/spring-boot/docs/current/reference/html/
2. **Oracle Database 21c Documentation** — https://docs.oracle.com/en/database/oracle/oracle-database/21/
3. **OWASP Top 10 (2021)** — https://owasp.org/Top10/
4. **PHP 8 Manual** — https://www.php.net/manual/en/
5. **Bootstrap 5 Documentation** — https://getbootstrap.com/docs/5.3/
6. **UML 2.5.1 Specification (OMG)** — https://www.omg.org/spec/UML/
7. **Réglementation BCEAO** — Instructions relatives à la lutte contre le blanchiment des capitaux et le financement du terrorisme dans les États membres de l'UEMOA.
8. **JUnit 5 User Guide** — https://junit.org/junit5/docs/current/user-guide/
9. **Martin Fowler, *Patterns of Enterprise Application Architecture*** — Addison-Wesley, 2002.
10. **Eric Evans, *Domain-Driven Design*** — Addison-Wesley, 2003.
11. **Craig Walls, *Spring in Action*** — Manning, 6e édition, 2022.
12. **Faker (Python)** — https://faker.readthedocs.io/en/master/
13. **cx_Oracle** — https://cx-oracle.readthedocs.io/en/latest/
14. **Git Flow** — https://nvie.com/posts/a-successful-git-branching-model/
15. **RFC 7519 — JSON Web Token (JWT)** — https://tools.ietf.org/html/

rfc7519